

JAVASCRIPT FOR WEB DEVELOPMENT

MASTER BOOK

Building Strong JavaScript Foundations for Real-World Web

Applications

Beginner to Advanced Guide for Students and Developers

Author: Ishfaq Ahmad Dar

Edition: First Edition – 2026

COPYRIGHT PAGE

Copyright © 2026 Ishfaq Ahmad Dar

All rights reserved.

No part of this book may be reproduced, distributed, or transmitted in any form or by any means, including photocopying, recording, or other electronic or mechanical methods, without prior written permission of the author, except in the case of brief quotations used in reviews or educational references.

This book is intended for educational purposes only. The author assumes no responsibility for any errors or omissions, or for any outcomes resulting from the use of the information provided in this book.

DEDICATION

This book is dedicated to all learners who are striving to build a better future through education and skill development.

To those who are starting from zero, facing challenges, and still choosing to learn every day — this work is for you.

Your consistency and effort will shape your success.

PREFACE

This book is designed to help learners master JavaScript for web development from beginner to advanced level in a structured and practical way.

The content focuses on building strong fundamentals first, then gradually moving toward real-world applications, projects, and interview preparation. Each chapter is written in simple language with clear explanations, practical examples, and exercises to reinforce learning.

The goal of this book is not only to teach syntax but to develop problem-solving skills and the ability to build real applications. By completing this book, a learner should be able to confidently work on web development projects and prepare for technical interviews.

Goal of This Book

To provide a complete, structured, and practical learning path that helps students:

- Understand JavaScript deeply
- Apply concepts in real-world scenarios
- Build strong problem-solving skills
- Prepare for technical interviews
- Become job-ready web developers

TABLE OF CONTENTS

PART 1: JAVASCRIPT FUNDAMENTALS

Chapter 1: Introduction to JavaScript

Chapter 2: Setting Up Environment

Chapter 3: Variables and Data Types

Chapter 4: Operators and Expressions

Chapter 5: Control Flow

PART 2: FUNCTIONS AND CORE CONCEPTS

Chapter 6: Functions in Depth

Chapter 7: Scope and Closures

Chapter 8: Hoisting

Chapter 9: Execution Context & Call Stack

PART 3: OBJECTS AND ARRAYS

Chapter 10: Objects in JavaScript

Chapter 11: Arrays and Methods

Chapter 12: Destructuring

Chapter 13: Spread and Rest Operators

PART 4: DOM MANIPULATION

Chapter 14: What is DOM

Chapter 15: Selecting Elements

Chapter 16: Event Handling

Chapter 17: DOM Project (Todo App)

PART 5: ASYNCHRONOUS JAVASCRIPT

Chapter 18: Callbacks

Chapter 19: Promises

Chapter 20: Async/Await

Chapter 21: Fetch API

PART 6: ADVANCED JAVASCRIPT

Chapter 22: Prototypes and Inheritance

Chapter 23: Classes (ES6)

Chapter 24: Modules

Chapter 25: Error Handling

PART 7: JAVASCRIPT IN REAL WEB DEVELOPMENT

Chapter 26: Working with APIs

Chapter 27: Form Handling & Validation

Chapter 28: Local Storage & Session Storage

Chapter 29: Debugging Techniques

PART 8: PROJECTS

Chapter 30: Calculator

Chapter 31: Todo App (Advanced)

Chapter 32: Weather App

Chapter 33: Mini E-commerce UI

PART 9: INTERVIEW PREPARATION

Chapter 34: JavaScript Interview Questions

Chapter 35: Coding Problems

Chapter 36: Scenario-Based Questions

CHAPTER 1: INTRODUCTION TO JAVASCRIPT

What is JavaScript

JavaScript is a high-level, interpreted programming language used to make web pages interactive and dynamic. It allows developers to control behavior on a webpage, respond to user actions, update content without reloading the page, and communicate with servers.

In web development, three core technologies work together:

- HTML → Structure of the webpage
- CSS → Styling and layout
- JavaScript → Behavior and logic

Example:

```
<!DOCTYPE html>
<html>
<head>
  <title>JavaScript Example</title>
</head>
<body>

<button onclick="showMessage()">Click Me</button>

<script>
function showMessage() {
  alert("Welcome to JavaScript!");
}
</script>
```

```
</body>  
</html>
```

Explanation:

A button is created using HTML. When the button is clicked, JavaScript executes the function `showMessage()`, which displays a popup message.

Output:

A popup alert appears with the text "Welcome to JavaScript!"

Why JavaScript is Important

JavaScript is essential for modern web development because:

- It enables interactivity (buttons, forms, animations)
- It allows real-time updates without reloading the page
- It communicates with servers using APIs
- It is supported by all modern browsers
- It is used in frontend and backend development (Node.js)

Real-world examples:

- Form validation before submission
- Live search suggestions
- Interactive dashboards
- Online games
- Chat applications

History and Evolution of JavaScript

- Created in 1995 by Brendan Eich at Netscape
- Initially named Mocha, then LiveScript, finally JavaScript
- Designed for quick scripting inside browsers
- Standardized as ECMAScript (ES)

Important versions:

- ES3 (1999): Stable version
- ES5 (2009): Introduced strict mode, JSON support
- ES6 (2015): Major upgrade
 - let and const
 - Arrow functions
 - Classes
 - Modules
- ES7 and later: Continuous improvements

How JavaScript Works in the Browser

When a webpage loads, the browser performs the following steps:

1. Loads HTML
2. Parses HTML and builds DOM (Document Object Model)
3. Loads CSS and applies styles
4. Executes JavaScript
5. Updates DOM dynamically

JavaScript interacts with the DOM to change content and structure.

Example:

```
<p id="demo">Hello</p>
```

```
<script>
```

```
document.getElementById("demo").innerText = "Hello, World!";  
</script>
```

Explanation:

JavaScript selects the paragraph using its ID and changes the text.

Output:

The text changes from "Hello" to "Hello, World!"

JavaScript Engine

A JavaScript engine is a program that executes JavaScript code.

Examples:

- Chrome → V8 Engine
- Firefox → SpiderMonkey
- Safari → JavaScriptCore

Working:

- Reads JavaScript code
- Converts it into machine code
- Executes it line by line

Ways to Use JavaScript in HTML

1. Inline JavaScript

```
<button onclick="alert('Hello')">Click</button>
```

Used directly inside HTML tags. Not recommended for large applications.

2. Internal JavaScript

```
<script>  
console.log("Hello from internal JS");  
</script>
```

Written inside `<script>` tag in HTML file.

3. External JavaScript

```
<script src="script.js"></script>
```

script.js:

```
console.log("Hello from external JS");
```

Best practice for real projects because it separates code from HTML.

JavaScript Output Methods

JavaScript provides different ways to display output:

1. alert()

```
alert("Hello");
```

Shows a popup message.

2. `console.log()`

```
console.log("Debug message");
```

Used for debugging in browser console.

3. `document.write()`

```
document.write("Hello");
```

Writes directly to webpage (not recommended in modern development).

4. `innerHTML` / `innerText`

```
document.getElementById("demo").innerHTML = "New Content";
```

Updates webpage content dynamically.

JavaScript Syntax Basics

- Case-sensitive language
- Statements end with semicolon (optional but recommended)
- Code blocks use curly braces { }

Example:

```
var name = "John";  
console.log(name);
```

Comments in JavaScript

Used to explain code or disable execution.

Single-line:

```
// This is a comment
```

Multi-line:

```
/* This is  
   a multi-line comment */
```

Key Features of JavaScript

- Interpreted language
- Dynamic typing
- Event-driven
- Object-oriented
- Functional programming support
- Platform independent

Real-World Example: Form Interaction

```
<input type="text" id="name" placeholder="Enter name">
<button onclick="displayName()">Submit</button>

<p id="result"></p>

<script>
function displayName() {
    let name = document.getElementById("name").value;
    document.getElementById("result").innerText = "Hello " +
name;
}
</script>
```

Explanation:

User enters a name. When button is clicked, JavaScript reads input value and displays greeting.

Output:

If user enters "Ali", output becomes "Hello Ali"

Common Mistakes

1. Confusing JavaScript with Java

They are completely different languages.

2. Forgetting to include script tag

Code will not run without `<script>`.

3. Using `document.write` in modern apps

It can overwrite entire page content.

4. Not checking console for errors

Use DevTools → Console for debugging.

5. Incorrect element selection

If ID or selector is wrong, code fails.

Interview Questions

1. What is JavaScript and where is it used?
2. Explain difference between JavaScript and Java.
3. What happens when a browser loads a webpage?
4. What is DOM?
5. What is a JavaScript engine?
6. Difference between inline, internal, and external JavaScript?
7. Why is JavaScript called dynamic language?

Practice Exercises

1. Write a JavaScript program to show an alert message "Hello World".
2. Create a button that changes text of a paragraph when clicked.
3. Write code to print your name in console.
4. Create an input field and display entered value on screen.
5. Add a button that shows current date and time using JavaScript.

Mini Practice Task

Create a webpage with:

- One input field
- One button
- One paragraph

When user clicks button, display:

"Welcome, [username]"

CHAPTER 2: SETTING UP ENVIRONMENT

Introduction

Before writing JavaScript programs, a proper development environment is required. This includes tools to write code, run code, and debug errors efficiently.

This chapter covers:

- Browser DevTools
- Setting up Visual Studio Code
- Project structure
- Running JavaScript code
- Introduction to Node.js

Browser Developer Tools (DevTools)

Every modern browser provides built-in tools to write, test, and debug JavaScript.

How to Open DevTools

- Press **F12**
- Right-click → Inspect
- Ctrl + Shift + I

Important DevTools Tabs

1. Console

Used to run JavaScript and debug code.

Example:

```
console.log("Hello from Console");
```

Output:

Message appears inside the console.

Use cases:

- Debugging
- Testing small code snippets
- Checking errors

2. Elements

- Shows HTML structure (DOM)
- Allows editing elements live

Example:

Change text directly and see updates instantly.

3. Network

- Shows API calls and resources
- Useful for debugging fetch/API requests

4. Sources

- Debug JavaScript files
- Add breakpoints

Writing JavaScript in Browser Console

You can directly write JavaScript in console without creating files.

Example:

```
let a = 5;  
let b = 10;  
console.log(a + b);
```

Output:

15

Setting Up Visual Studio Code (VS Code)

VS Code is a lightweight and powerful code editor widely used for JavaScript development.

Installation Steps

1. Download from official website
2. Install and open
3. Create a folder for your project

Recommended Extensions

1. Live Server

- Runs HTML files in browser
- Automatically reloads when file changes

2. Prettier

- Formats code automatically
- Improves readability

3. JavaScript (ES6) Snippets

- Provides shortcuts for writing code

Project Folder Structure

Example:

```
project/  
|  
├── index.html  
├── script.js  
└── style.css
```

Connecting JavaScript File

Inside HTML:

```
<script src="script.js"></script>
```

Best practice:

- Place script before closing `</body>` tag

Writing First JavaScript File

script.js:

```
console.log("JavaScript is working");
```

Running JavaScript in Browser

Steps:

1. Open index.html
2. Right-click → Open with Live Server
3. Open DevTools → Console
4. See output

Example: Full Setup

index.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Setup Test</title>
</head>
```



```
<body>

<h1 id="heading">Hello</h1>

<script src="script.js"></script>

</body>
</html>
```

script.js

```
document.getElementById("heading").innerText = "Environment Ready";
```

Explanation:

JavaScript changes text inside <h1> element.

Output:

"Hello" becomes "Environment Ready"

Introduction to Node.js

Node.js allows running JavaScript outside the browser.

Why Node.js

- Run backend applications

- Execute JavaScript on server
- Build APIs

Install Node.js

1. Download and install
2. Open terminal
3. Check version:

```
node -v
```

Running JavaScript using Node

Create file:

```
console.log("Running with Node.js");
```

Run in terminal:

```
node script.js
```

Output:

Running with Node.js

Differences: Browser vs Node.js

Feature	Browser	Node.js
Environment	Frontend	Backend
DOM Access	Yes	No
API Usage	Limited	Full access

File System	No	Yes
-------------	----	-----

Common Mistakes

1. Not linking JavaScript file correctly

Incorrect path causes code not to run

2. Placing script in wrong position

If placed before HTML loads, elements may not exist

3. Forgetting to open console

No visibility of output/errors

4. Not saving file before running

Changes will not reflect

5. Mixing Node and browser code

Some features work only in browser (DOM)

Interview Questions

1. What is DevTools and why is it used?
2. What is console.log()?
3. How do you link JavaScript to HTML?
4. What is Live Server?
5. What is Node.js?
6. Difference between Node.js and browser JavaScript?
7. Why is script placed at bottom of HTML?

Practice Exercises

1. Open DevTools and print "Hello World" in console
2. Create a project folder with:
 - a. index.html
 - b. script.js
3. Link script.js and print a message
4. Change text of a heading using JavaScript
5. Install Node.js and run a simple JavaScript file

Mini Practice Task

Create a webpage:

- One heading
- One button

When button is clicked:

- Change heading text to "Setup Complete"

CHAPTER 3: VARIABLES AND DATA TYPES

Introduction

Variables are used to store data in a program. Data types define what kind of data is stored in a variable. Understanding variables and data types is essential because every JavaScript program depends on storing and manipulating data.

What is a Variable

A variable is a container used to store values. These values can be numbers, text, true/false values, or more complex structures.

Example:

```
let name = "Ali";  
console.log(name);
```

Explanation:

A variable named `name` stores the string "Ali" and is printed in the console.

Output:

Ali

Declaring Variables

JavaScript provides three ways to declare variables:

- var
- let
- const

var

```
var age = 25;  
console.log(age);
```

Features of var:

- Function scoped
- Can be redeclared
- Can be updated
- Hoisted (initialized with undefined)

Example:

```
var x = 10;  
var x = 20;  
console.log(x);
```

Output:

20

let

```
let city = "Delhi";  
console.log(city);
```

Features of let:

- Block scoped
- Cannot be redeclared in same scope
- Can be updated

Example:

```
let x = 10;  
x = 15;  
console.log(x);
```

Output:

15

const

```
const pi = 3.14;  
console.log(pi);
```

Features of const:

- Block scoped
- Cannot be redeclared
- Cannot be updated
- Must be initialized at declaration

Example:

```
const a = 10;  
// a = 20; // Error
```

Difference Between var, let, const

Feature	var	let	const
Scope	Function	Block	Block
Redeclaration	Yes	No	No
Reassignment	Yes	Yes	No
Hoisting	Yes	Yes (TDZ)	Yes (TDZ)

Naming Rules for Variables

- Must start with letter, _ or \$
- Cannot start with number
- No spaces allowed
- Case-sensitive

Valid:

```
let userName = "Ali";  
let _value = 10;
```

Invalid:

```
// let 1name = "Ali";  
// let user name = "Ali";
```

Data Types in JavaScript

JavaScript has two main categories:

1. Primitive Data Types
2. Non-Primitive Data Types

Primitive Data Types

1. Number

Represents numeric values.

```
let num = 100;  
console.log(num);
```

Output:

100

2. String

Represents text.

```
let message = "Hello";  
console.log(message);
```

Output:

Hello

3. Boolean

Represents true or false.

```
let isLoggedIn = true;  
console.log(isLoggedIn);
```

Output:

true

4. Undefined

A variable declared but not assigned value.

```
let x;  
console.log(x);
```

Output:

undefined

5. Null

Represents intentional empty value.

```
let data = null;  
console.log(data);
```

Output:

null

6. Symbol (Basic Idea)

Used for unique identifiers.

```
let id = Symbol("id");
```

Non-Primitive Data Types

1. Object

Collection of key-value pairs.

```
let person = {  
  name: "Ali",  
  age: 25  
};  
console.log(person.name);
```

Output:

Ali

2. Array

Stores multiple values.

```
let fruits = ["Apple", "Banana", "Mango"];  
console.log(fruits[1]);
```

Output:

Banana

Dynamic Typing

JavaScript is dynamically typed, meaning variable type can change.

```
let value = 10;  
value = "Hello";  
console.log(value);
```

Output:

Hello

typeof Operator

Used to check data type.

```
let x = 10;  
console.log(typeof x);
```

Output:

number

Examples:

```
console.log(typeof "Hello");    // string  
console.log(typeof true);       // boolean  
console.log(typeof null);       // object (special case)
```

Type Conversion

Implicit Conversion

```
let result = "5" + 2;  
console.log(result);
```

Output:

"52"

Explicit Conversion

```
let num = Number("10");  
console.log(num);
```

Output:

10

Real-World Example

```
let price = 100;  
let quantity = 2;  
let total = price * quantity;  
  
console.log("Total:", total);
```

Output:

Total: 200

Common Mistakes

1. Using var instead of let/const

Leads to scope issues

2. Reassigning const variables

Causes error

3. Confusing null and undefined

They represent different meanings

4. Incorrect type checking

typeof null returns "object"

5. Not initializing variables

May lead to undefined errors

Interview Questions

1. What is a variable in JavaScript?
2. Difference between var, let, const?
3. What are primitive data types?
4. What is dynamic typing?
5. Difference between null and undefined?
6. What is typeof operator?
7. Can const variables change?

Practice Exercises

1. Declare variables using var, let, and const
2. Store your name, age, and isStudent in variables
3. Create an object with your details
4. Create an array of 5 numbers
5. Use typeof to check data types

Mini Practice Task

Create a program:

- Store product name, price, and quantity
- Calculate total price
- Print result in console

CHAPTER 4: OPERATORS AND EXPRESSIONS

Introduction

Operators are symbols used to perform operations on values and variables.

Expressions are combinations of values, variables, and operators that produce a result.

Example:

```
let result = 10 + 5;
```

Explanation:

`10 + 5` is an expression. `+` is an operator.

Output:

15

Types of Operators in JavaScript

- Arithmetic Operators
- Assignment Operators
- Comparison Operators
- Logical Operators
- Unary Operators
- Ternary Operator

Arithmetic Operators

Used for mathematical calculations.

Operator	Meaning	Example
+	Addition	5 + 2
-	Subtraction	5 - 2
*	Multiplication	5 * 2
/	Division	10 / 2
%	Modulus	5 % 2
**	Exponent	2 ** 3

Examples

```
let a = 10;
let b = 3;

console.log(a + b); // 13
console.log(a - b); // 7
console.log(a * b); // 30
console.log(a / b); // 3.33
console.log(a % b); // 1
console.log(a ** 2); // 100
```

Assignment Operators

Used to assign values to variables.

Operator	Example	Meaning
=	x = 5	Assign value
+=	x += 2	x = x + 2
-=	x -= 2	x = x - 2

<code>*</code>	<code>x *= 2</code>	<code>x = x * 2</code>
<code>/</code>	<code>x /= 2</code>	<code>x = x / 2</code>

Example

```
let x = 10;

x += 5;
console.log(x); // 15

x *= 2;
console.log(x); // 30
```

Comparison Operators

Used to compare values. Returns true or false.

Operator	Meaning
<code>==</code>	Equal (loose)
<code>===</code>	Strict equal
<code>!=</code>	Not equal
<code>!==</code>	Strict not equal
<code>></code>	Greater than
<code><</code>	Less than
<code>>=</code>	Greater or equal
<code><=</code>	Less or equal

Example

```
console.log(5 == "5"); // true
console.log(5 === "5"); // false
console.log(10 > 5); // true
```

```
console.log(3 <= 2);    // false
```

Explanation:

== compares value only, while === compares value and type.

Logical Operators

Used to combine conditions.

Operator	Meaning
&&	AND

!	NOT
---	-----

Example

```
let age = 20;
```

```
console.log(age > 18 && age < 30); // true
console.log(age < 18 || age > 25); // false
console.log(!(age > 18));          // false
```

Unary Operators

Operate on a single operand.

Operator	Meaning
+	Convert to number
-	Negation
++	Increment

-- Decrement

Example

```
let x = 5;

x++;
console.log(x); // 6

x--;
console.log(x); // 5
```

Increment Types

Post Increment

```
let x = 5;
console.log(x++); // 5
console.log(x);   // 6
```

Pre Increment

```
let x = 5;
console.log(++x); // 6
```

Ternary Operator

Short form of if-else.

Syntax:

```
condition ? value1 : value2;
```

Example

```
let age = 18;  
  
let result = (age >= 18) ? "Adult" : "Minor";  
console.log(result);
```

Output:

Adult

Operator Precedence

Operators are executed based on priority.

Example:

```
let result = 10 + 5 * 2;  
console.log(result);
```

Output:

20

Explanation:

Multiplication happens before addition.

Use brackets to control order:

```
let result = (10 + 5) * 2;  
console.log(result);
```

Output:

30

Expressions in JavaScript

An expression is any valid combination that produces a value.

Examples:

```
5 + 10  
x * y  
true && false
```

Real-World Example

```
let price = 200;  
let discount = 10;  
  
let finalPrice = price - (price * discount / 100);  
  
console.log(finalPrice);
```

Output:

180

Common Mistakes

1. Using `==` instead of `===`

May cause unexpected results

2. Forgetting operator precedence

Leads to wrong calculations

3. Misusing increment operators

Confusing pre and post increment

4. Using assignment (`=`) instead of comparison (`==` or `===`)
5. Ignoring type conversion in expressions

Interview Questions

1. What is an operator?
2. Difference between `==` and `===`?
3. What is operator precedence?
4. What are logical operators?
5. Explain ternary operator.
6. Difference between pre and post increment?

Practice Exercises

1. Perform all arithmetic operations on two numbers
2. Use assignment operators and update values
3. Compare two numbers using different operators
4. Write a program using logical operators
5. Convert if-else into ternary operator

Mini Practice Task

Create a program:

- Take marks as input
- If marks $\geq 50 \rightarrow$ "Pass"
- Else $\rightarrow$ "Fail"
- Use ternary operator

Now write Chapter 5: Control Flow in full detail following the same rules.

CHAPTER 5: CONTROL FLOW

Introduction

Control flow determines how a program executes step by step. It allows decision-making and repetition of tasks based on conditions.

Without control flow, a program runs line by line without any logic or flexibility.

Main types:

- Conditional statements (decision making)
- Loops (repetition)

Conditional Statements

Conditional statements allow the program to make decisions.

if Statement

Executes code only if condition is true.

Syntax:

```
if (condition) {  
    // code  
}
```

Example:

```
let age = 20;

if (age >= 18) {
    console.log("Eligible to vote");
}
```

Output:

Eligible to vote

if-else Statement

Executes one block if condition is true, otherwise another block.

```
let age = 16;

if (age >= 18) {
    console.log("Adult");
} else {
    console.log("Minor");
}
```

Output:

Minor

else-if Ladder

Used when multiple conditions are checked.

```
let marks = 75;
```

```
if (marks >= 90) {  
    console.log("A Grade");  
} else if (marks >= 60) {  
    console.log("B Grade");  
} else if (marks >= 40) {  
    console.log("C Grade");  
} else {  
    console.log("Fail");  
}
```

Output:

B Grade

Nested if

An if inside another if.

```
let age = 25;  
let hasID = true;  
  
if (age >= 18) {  
    if (hasID) {  
        console.log("Allowed");  
    }  
}
```

Output:

Allowed

switch Statement

Used when multiple conditions depend on a single value.

Syntax:

```
switch (value) {  
    case x:  
        // code  
        break;  
    default:  
        // code  
}
```

Example:

```
let day = 2;  
  
switch (day) {  
    case 1:  
        console.log("Monday");  
        break;  
    case 2:  
        console.log("Tuesday");  
        break;  
    case 3:  
        console.log("Wednesday");  
        break;  
    default:  
        console.log("Invalid");  
}
```

Output:

Tuesday

Important Notes on switch

- break stops execution
- Without break, all cases below will run
- default runs if no case matches

Loops

Loops are used to repeat code multiple times.

for Loop

Used when number of iterations is known.

Syntax:

```
for (initialization; condition; update) {  
    // code  
}
```

Example:

```
for (let i = 1; i <= 5; i++) {  
    console.log(i);  
}
```

Output:

1 2 3 4 5

while Loop

Runs while condition is true.

```
let i = 1;

while (i <= 5) {
    console.log(i);
    i++;
}
```

Output:

1 2 3 4 5

do-while Loop

Runs at least once, even if condition is false.

```
let i = 1;

do {
    console.log(i);
    i++;
} while (i <= 5);
```

Output:

1 2 3 4 5

break Statement

Stops loop immediately.

```
for (let i = 1; i <= 5; i++) {  
  if (i === 3) {  
    break;  
  }  
  console.log(i);  
}
```

Output:

1 2

continue Statement

Skips current iteration.

```
for (let i = 1; i <= 5; i++) {  
  if (i === 3) {  
    continue;  
  }  
  console.log(i);  
}
```

Output:

1 2 4 5

Real-World Example

Example: Login Check

```
let username = "admin";  
let password = "1234";  
  
if (username === "admin" && password === "1234") {  
    console.log("Login successful");  
} else {  
    console.log("Invalid credentials");  
}
```

Output:

Login successful

Example: Sum of Numbers

```
let sum = 0;  
  
for (let i = 1; i <= 5; i++) {  
    sum += i;  
}  
  
console.log(sum);
```

Output:

15

Common Mistakes

1. Using = instead of == or ===

Leads to incorrect conditions

2. Infinite loops

Forgetting update condition

3. Missing break in switch

Causes unexpected outputs

4. Wrong loop condition

Loop may not run or run infinitely

5. Confusing while and do-while

Interview Questions

1. What is control flow?
2. Difference between if and switch?
3. Difference between for and while loop?
4. What is infinite loop?
5. What is break and continue?
6. When do you use do-while loop?

Practice Exercises

1. Check if a number is positive or negative
2. Print numbers from 1 to 10 using loop
3. Find sum of first 10 numbers

4. Print even numbers between 1 to 20
5. Create a simple calculator using switch

Mini Practice Task

Create a program:

- Take a number
- If number is divisible by 2 → "Even"
- Else → "Odd"

Challenge Task

Create a menu-driven program using switch:

1. Add
2. Subtract
3. Multiply
4. Exit

CHAPTER 6: FUNCTIONS IN DEPTH

Introduction

Functions are reusable blocks of code that perform a specific task. Instead of writing the same code multiple times, functions allow you to define it once and reuse it whenever needed.

Functions help in:

- Code reusability
- Better organization
- Easier debugging
- Modular programming

What is a Function

A function is a block of code that runs when it is called.

Example:

```
function greet() {  
    console.log("Hello");  
}
```

```
greet();
```

Explanation:

The function `greet()` is defined and then called.

Output:

Hello

Function Declaration

A function defined using the `function` keyword.

```
function add(a, b) {  
    return a + b;  
}  
  
console.log(add(5, 3));
```

Output:

8

Function Expression

A function stored inside a variable.

```
let multiply = function(a, b) {  
    return a * b;  
};  
  
console.log(multiply(4, 3));
```

Output:

12

Difference: Declaration vs Expression

Feature	Declaration	Expression
Hoisted	Yes	No
Usage	Before defining	After defining

Arrow Functions

Short syntax introduced in ES6.

```
let subtract = (a, b) => {  
    return a - b;  
};  
  
console.log(subtract(10, 4));
```

Output:

6

Short Arrow Function

```
let square = x => x * x;  
  
console.log(square(5));
```

Output:

25

Parameters and Arguments

- Parameters → variables in function definition
- Arguments → values passed during function call

Example:

```
function greet(name) {  
    console.log("Hello " + name);  
}  
  
greet("Ali");
```

Output:

Hello Ali

Default Parameters

Used when no value is passed.

```
function greet(name = "Guest") {  
    console.log("Hello " + name);  
}  
  
greet();
```

Output:

Hello Guest

Return Values

Functions can return results using `return`.

```
function sum(a, b) {  
    return a + b;  
}  
  
let result = sum(3, 7);  
console.log(result);
```

Output:

10

Function Without Return

```
function display() {  
    console.log("No return value");  
}  
  
display();
```

Output:

No return value

Pure vs Impure Functions

Pure Function

- Same input → same output
- No side effects

```
function add(a, b) {  
    return a + b;  
}
```

Impure Function

- Depends on external data

```
let total = 0;  
  
function addToTotal(x) {  
    total += x;  
}
```

Callback Functions

A function passed as an argument to another function.

```
function greet(name, callback) {  
    console.log("Hello " + name);  
    callback();  
}  
  
function sayBye() {
```



```
        console.log("Goodbye");
    }

    greet("Ali", sayBye);
```

Output:

Hello Ali

Goodbye

Anonymous Functions

Functions without a name.

```
setTimeout(function() {
    console.log("Hello after 2 seconds");
}, 2000);
```

Immediately Invoked Function Expression (IIFE)

Function that runs immediately.

```
(function() {
    console.log("IIFE executed");
})();
```

Output:

IIFE executed

Real-World Example

Example: Discount Calculator

```
function calculateDiscount(price, discount) {  
    return price - (price * discount / 100);  
}  
  
let finalPrice = calculateDiscount(1000, 10);  
console.log(finalPrice);
```

Output:

900

Common Mistakes

1. Forgetting return statement

Function returns undefined

2. Confusing parameters and arguments
3. Not calling function after defining
4. Using arrow functions incorrectly with multiple lines
5. Overwriting function names

Interview Questions

1. What is a function in JavaScript?
2. Difference between function declaration and expression?
3. What are arrow functions?
4. What is a callback function?

5. What is a pure function?
6. What is IIFE?
7. What happens if function has no return?

Practice Exercises

1. Write a function to add two numbers
2. Write a function to check even/odd
3. Create a function with default parameter
4. Convert normal function into arrow function
5. Write a callback function example

Mini Practice Task

Create a function:

- Input: marks
- Output: grade
 - = 90 → A
 - = 60 → B
 - else → Fail

Challenge Task

Create a calculator using functions:

- add()
- subtract()
- multiply()
- divide()

Call functions based on user input.

CHAPTER 7: SCOPE AND CLOSURES

Introduction

Scope determines where variables are accessible in a program. Closures allow functions to remember variables from their outer scope even after that scope has finished execution.

Understanding scope and closures is important for writing predictable, bug-free, and optimized JavaScript code.

What is Scope

Scope defines the visibility and accessibility of variables.

Types of scope in JavaScript:

- Global Scope
- Function Scope
- Block Scope

Global Scope

A variable declared outside any function or block is in global scope. It can be accessed anywhere in the program.

```
let name = "Ali";

function greet() {
    console.log(name);
}
```

```
greet();
```

Output:

Ali

Function Scope

Variables declared inside a function are accessible only within that function.

```
function test() {  
    let x = 10;  
    console.log(x);  
}  
  
test();  
// console.log(x); // Error
```

Explanation:

x cannot be accessed outside the function.

Block Scope

Variables declared with `let` and `const` inside a block `{ }` are accessible only within that block.

```
if (true) {  
    let x = 5;  
    console.log(x);  
}
```

```
// console.log(x); // Error
```

var vs let (Scope Difference)

```
if (true) {  
    var a = 10;  
    let b = 20;  
}  
  
console.log(a); // Works  
// console.log(b); // Error
```

Explanation:

var is function-scoped, **let** is block-scoped.

Lexical Scope

Lexical scope means inner functions can access variables from their outer (parent) functions.

```
function outer() {  
    let x = 10;  
  
    function inner() {  
        console.log(x);  
    }  
  
    inner();  
}
```

```
outer();
```

Output:

10

Scope Chain

JavaScript searches for variables from inner scope to outer scope.

```
let a = 1;
```

```
function outer() {  
    let b = 2;  
  
    function inner() {  
        let c = 3;  
        console.log(a, b, c);  
    }  
  
    inner();  
}
```

```
outer();
```

Output:

1 2 3

What is Closure

A closure is created when a function remembers variables from its outer scope even after the outer function has finished execution.

Closure Example

```
function outer() {  
    let count = 0;  
  
    return function inner() {  
        count++;  
        console.log(count);  
    };  
}
```

```
let counter = outer();
```

```
counter();  
counter();  
counter();
```

Output:

1

2

3

Explanation:

The inner function remembers `count` even after `outer()` has finished.

Another Closure Example

```
function greet(name) {  
    return function() {  
        console.log("Hello " + name);  
    };  
}
```

```
let sayHello = greet("Ali");  
sayHello();
```

Output:

Hello Ali

Real-World Use of Closures

Example: Private Variable

```
function createCounter() {  
    let count = 0;  
  
    return {  
        increment: function() {  
            count++;  
            console.log(count);  
        },  
        decrement: function() {  
            count--;  
            console.log(count);  
        }  
    };  
}
```

```
let counter = createCounter();
```

```
counter.increment();  
counter.increment();  
counter.decrement();
```

Output:

1

2

1

Explanation:

count is protected and cannot be accessed directly.

Closures in Event Handling

```
function setupButton() {  
    let message = "Button clicked";  
  
    document.getElementById("btn").onclick = function() {  
        console.log(message);  
    };  
}
```

Explanation:

The function remembers `message` even after setup is done.

Advantages of Closures

- Data privacy
- Persistent state
- Useful in callbacks and event handling
- Used in functional programming

Common Mistakes

1. Confusing scope with lifetime

Scope defines access, not how long variable lives

2. Using var inside loops

Can cause unexpected behavior

3. Forgetting closures capture reference, not value
4. Overusing global variables
5. Not understanding lexical scope

Interview Questions

1. What is scope in JavaScript?
2. Difference between global and local scope?
3. What is block scope?
4. What is lexical scope?
5. What is closure?
6. Why are closures useful?
7. Give real-world example of closure

Practice Exercises

1. Create a function with local variable and try accessing outside
2. Demonstrate block scope using let
3. Write a nested function example
4. Create a closure that counts number of calls
5. Create a function that returns greeting using closure

Mini Practice Task

Create a function:

- It returns another function
- The inner function increments a counter
- Each call prints updated value

Challenge Task

Create a bank account system using closure:

- Private balance
- deposit() function
- withdraw() function
- balance should not be directly accessible

CHAPTER 8: HOISTING

Introduction

Hoisting is JavaScript's default behavior of moving variable and function declarations to the top of their scope before code execution.

This does not mean the code physically moves. The JavaScript engine processes declarations first during the creation phase, then executes code line by line.

Understanding hoisting helps avoid unexpected bugs and errors.

How JavaScript Executes Code

JavaScript runs in two phases:

1. Creation Phase
 - a. Memory is allocated
 - b. Variables are declared
 - c. Functions are stored
2. Execution Phase
 - a. Code runs line by line
 - b. Values are assigned

Hoisting with var

Variables declared with `var` are hoisted and initialized with `undefined`.

```
console.log(x);  
var x = 10;
```

Output:

undefined

Explanation:

JavaScript treats it as:

```
var x;  
console.log(x);  
x = 10;
```

Hoisting with let

Variables declared with `let` are hoisted but not initialized.

```
console.log(x);  
let x = 10;
```

Output:

Error

Explanation:

The variable exists but cannot be accessed before initialization.

Hoisting with const

Same behavior as `let`.

```
console.log(a);  
const a = 5;
```

Output:

Error

Temporal Dead Zone (TDZ)

The time between variable declaration and initialization is called Temporal Dead Zone.

During this period:

- Variables exist
- But cannot be accessed

Example:

```
{  
    console.log(x);  
    let x = 10;  
}
```

Output:

Error

Hoisting with Functions

Function Declaration

Fully hoisted.

```
greet();
```

```
function greet() {  
    console.log("Hello");  
}
```



```
}
```

Output:

Hello

Function Expression

Not hoisted like function declaration.

```
greet();

var greet = function() {
  console.log("Hello");
};
```

Output:

Error

Explanation:

Only variable is hoisted, not the function.

Arrow Function Hoisting

Behaves like function expressions.

```
sayHi();

let sayHi = () => {
  console.log("Hi");
};
```

```
};
```

Output:

Error

Hoisting Summary

Type	Hoisted	Initialized	Accessible Before Declaration
var	Yes	undefined	Yes
let	Yes	No	No (TDZ)
const	Yes	No	No (TDZ)
Function Declaration	Yes	Yes	Yes
Function Expression	Partial	No	No

Real-World Example

```
function calculate() {  
    console.log(total);  
    var total = 100;  
}
```

```
calculate();
```

Output:

undefined

Explanation:

`var total` is hoisted and initialized as undefined.

Example with let

```
function calculate() {  
    console.log(total);  
    let total = 100;  
}  
  
calculate();
```

Output:

Error

Common Mistakes

1. Accessing variables before declaration

Causes undefined or error

2. Confusing var and let behavior
3. Using function expressions before definition
4. Ignoring Temporal Dead Zone
5. Writing code in wrong order

Interview Questions

1. What is hoisting?
2. How does var behave in hoisting?
3. What is Temporal Dead Zone?
4. Difference between let and var in hoisting?
5. Are functions hoisted?
6. Why function declarations work before definition?

Practice Exercises

1. Predict output of:

```
console.log(a);  
var a = 5;
```

2. Predict output of:

```
console.log(a);  
let a = 5;
```

3. Call a function before defining it
4. Convert function declaration to expression and test
5. Write example showing TDZ

Mini Practice Task

Create a program:

- Declare variable using var
- Print before and after assignment
- Observe behavior

Challenge Task

Write code:

- Use let and const
- Try accessing before declaration

- Explain errors

CHAPTER 9: EXECUTION CONTEXT & CALL STACK

Introduction

Execution Context and Call Stack explain how JavaScript runs code internally. They help understand function calls, variable storage, and execution order.

Understanding these concepts is important for debugging, optimizing code, and handling complex programs.

What is Execution Context

Execution Context is an environment where JavaScript code is executed.

Types of execution context:

- Global Execution Context (GEC)
- Function Execution Context (FEC)

Global Execution Context (GEC)

- Created when the program starts
- Only one global context exists
- Contains global variables and functions

Example:

```
let x = 10;
```

```
function greet() {  
    console.log("Hello");  
}  
  
greet();
```

Explanation:

The global context stores `x` and `greet()`.

Function Execution Context (FEC)

- Created whenever a function is called
- Each function gets its own context

Example:

```
function add(a, b) {  
    return a + b;  
}  
  
add(2, 3);
```

Explanation:

When `add()` is called, a new execution context is created.

Phases of Execution Context

Each execution context has two phases:

1. Creation Phase

- Memory allocation
- Variables initialized as undefined
- Functions stored

2. Execution Phase

- Code runs line by line
- Values are assigned
- Functions are executed

Example: Execution Phases

```
console.log(a);
```

```
var a = 5;
```

Creation Phase:

- $a \rightarrow \text{undefined}$

Execution Phase:

- $\text{console.log}(a) \rightarrow \text{undefined}$
- $a = 5$

Output:

undefined

What is Call Stack

Call Stack is a data structure used to manage execution contexts.

- Follows LIFO (Last In, First Out)
- Keeps track of function calls

Call Stack Working

Example:

```
function first() {  
    second();  
}  
  
function second() {  
    third();  
}  
  
function third() {  
    console.log("Hello");  
}  
  
first();
```

Step-by-Step Execution

1. Global Execution Context pushed
2. first() pushed
3. second() pushed
4. third() pushed
5. console.log runs

6. third() removed
7. second() removed
8. first() removed

Visual Flow

Call Stack:

Top → third()

second()

first()

Bottom → Global

Stack Overflow

Occurs when too many function calls are added without returning.

Example:

```
function loop() {  
    loop();  
}
```

```
loop();
```

Output:

Error (Maximum call stack size exceeded)

Real-World Example

```
function calculate() {  
    let result = add(5, 3);  
    console.log(result);  
}  
  
function add(a, b) {  
    return a + b;  
}  
  
calculate();
```

Execution Flow:

- Global context created
- calculate() called
- add() called
- result returned
- printed

Output:

8

Execution Context + Call Stack Combined

```
let x = 1;  
  
function one() {  
    let y = 2;  
    two();  
}
```

```
function two() {  
    let z = 3;  
    console.log(x + z);  
}
```

one();

Output:

4

Explanation:

- two() accesses x from global scope
- Uses its own variable z

Common Mistakes

1. Not understanding execution order

Leads to confusion in nested functions

2. Infinite recursion

Causes stack overflow

3. Misunderstanding variable scope in functions
4. Ignoring creation phase

Interview Questions

1. What is execution context?

2. Types of execution context?
3. What happens in creation phase?
4. What is call stack?
5. What is stack overflow?
6. How does JavaScript execute nested functions?

Practice Exercises

1. Write a program with 3 nested functions and trace execution
2. Predict output:

```
function test() {  
    console.log("A");  
}
```

```
test();
```

3. Create recursive function and observe behavior
4. Trace call stack manually
5. Write function calling another function

Mini Practice Task

Create program:

- function A calls B
- B calls C
- C prints a message

Trace call stack manually

Challenge Task

Write recursive function:

- Print numbers from 5 to 1
- Track call stack behavior

CHAPTER 10: OBJECTS IN JAVASCRIPT

Introduction

Objects are one of the most important concepts in JavaScript. They are used to store data in key-value pairs and represent real-world entities.

Examples:

- A user (name, age, email)
- A product (name, price, category)
- A car (brand, model, speed)

What is an Object

An object is a collection of properties.

Each property has:

- Key (name)
- Value

Example:

```
let person = {  
  name: "Ali",  
  age: 25,  
  city: "Delhi"  
};  
  
console.log(person);
```

Output:

```
{ name: "Ali", age: 25, city: "Delhi" }
```

Accessing Object Properties

Dot Notation

```
console.log(person.name);
```

Output:

Ali

Bracket Notation

```
console.log(person["age"]);
```

Output:

25

Difference: Dot vs Bracket

Feature	Dot Notation	Bracket Notation
Syntax	simple	flexible
Dynamic keys	No	Yes

Example:


```
let key = "city";  
console.log(person[key]);
```

Output:

Delhi

Adding Properties

```
person.country = "India";  
console.log(person);
```

Updating Properties

```
person.age = 30;  
console.log(person.age);
```

Output:

30

Deleting Properties

```
delete person.city;  
console.log(person);
```

Object Methods

Functions inside objects are called methods.

```
let user = {  
  name: "Ali",  
  greet: function() {  
    console.log("Hello " + this.name);  
  }  
};  
  
user.greet();
```

Output:

Hello Ali

this Keyword

Refers to the current object.

```
let car = {  
  brand: "BMW",  
  show: function() {  
    console.log(this.brand);  
  }  
};  
  
car.show();
```

Output:

BMW

Object Creation Methods

1. Object Literal (Most Common)

```
let student = {  
  name: "Sara",  
  marks: 90  
};
```

2. Using new Object()

```
let obj = new Object();  
obj.name = "Ali";  
obj.age = 22;
```

3. Constructor Function

```
function Person(name, age) {  
  this.name = name;  
  this.age = age;  
}  
  
let p1 = new Person("Ali", 25);  
console.log(p1.name);
```

Output:

Ali

Nested Objects

Objects inside objects.

```
let student = {  
  name: "Ali",  
  address: {  
    city: "Delhi",  
    pin: 12345  
  }  
};  
  
console.log(student.address.city);
```

Output:

Delhi

Object.keys(), values(), entries()

```
let user = {  
  name: "Ali",  
  age: 25  
};  
  
console.log(Object.keys(user));  
console.log(Object.values(user));  
console.log(Object.entries(user));
```

Output:

```
["name", "age"]
```

```
["Ali", 25]
```

```
[["name", "Ali"], ["age", 25]]
```

Looping Through Objects

```
for (let key in user) {  
    console.log(key, user[key]);  
}
```

Output:

```
name Ali
```

```
age 25
```

Real-World Example

```
let product = {  
    name: "Laptop",  
    price: 50000,  
    quantity: 2,  
    total: function() {  
        return this.price * this.quantity;  
    }  
};  
  
console.log(product.total());
```

Output:

100000

Shallow Copy of Object

```
let obj1 = { a: 1 };  
let obj2 = obj1;
```

```
obj2.a = 5;
```

```
console.log(obj1.a);
```

Output:

5

Explanation:

Both refer to same memory.

Copy Using Spread Operator

```
let obj1 = { a: 1 };  
let obj2 = { ...obj1 };
```

```
obj2.a = 5;
```

```
console.log(obj1.a);
```

Output:

Common Mistakes

1. Using = instead of : in object
2. Forgetting quotes in bracket notation
3. Confusing this keyword
4. Modifying original object accidentally
5. Not checking undefined properties

Interview Questions

1. What is an object in JavaScript?
2. Difference between dot and bracket notation?
3. What is this keyword?
4. How to loop through object?
5. What is constructor function?
6. Difference between shallow and deep copy?

Practice Exercises

1. Create an object for a student
2. Add, update, and delete properties
3. Create a method inside object
4. Loop through object and print keys
5. Copy object using spread operator

Mini Practice Task

Create an object:

- name
- price
- quantity

Add method to calculate total price

Challenge Task

Create a user system:

- Store user details in object
- Add login method
- Display user info using method

CHAPTER 11: ARRAYS AND METHODS

Introduction

Arrays are used to store multiple values in a single variable. They are ordered collections, meaning each value has an index (position).

Arrays are widely used in real-world applications such as:

- Storing lists of users
- Managing products
- Handling API data

What is an Array

An array is a collection of elements stored in a single variable.

```
let fruits = ["Apple", "Banana", "Mango"];  
console.log(fruits);
```

Output:

```
["Apple", "Banana", "Mango"]
```

Accessing Array Elements

Each element has an index starting from 0.

```
let fruits = ["Apple", "Banana", "Mango"];
```

```
console.log(fruits[0]); // Apple  
console.log(fruits[1]); // Banana
```

Modifying Array Elements

```
let fruits = ["Apple", "Banana", "Mango"];  
  
fruits[1] = "Orange";  
console.log(fruits);
```

Output:

```
["Apple", "Orange", "Mango"]
```

Array Length

```
let numbers = [1, 2, 3, 4];  
  
console.log(numbers.length);
```

Output:

```
4
```

Common Array Methods

push()

Adds element at the end.

```
let arr = [1, 2];
```

```
arr.push(3);  
console.log(arr);
```

Output:

```
[1, 2, 3]
```

pop()

Removes last element.

```
let arr = [1, 2, 3];
```

```
arr.pop();  
console.log(arr);
```

Output:

```
[1, 2]
```

shift()

Removes first element.

```
let arr = [1, 2, 3];
```

```
arr.shift();  
console.log(arr);
```

Output:

```
[2, 3]
```

unshift()

Adds element at beginning.

```
let arr = [2, 3];  
  
arr.unshift(1);  
console.log(arr);
```

Output:

```
[1, 2, 3]
```

indexOf()

Finds index of element.

```
let arr = ["a", "b", "c"];  
  
console.log(arr.indexOf("b"));
```

Output:

```
1
```

includes()

Checks if value exists.

```
let arr = [1, 2, 3];  
  
console.log(arr.includes(2));
```

Output:

true

slice()

Returns a portion of array (no change to original).

```
let arr = [1, 2, 3, 4];  
  
let newArr = arr.slice(1, 3);  
console.log(newArr);
```

Output:

[2, 3]

splice()

Adds/removes elements.

```
let arr = [1, 2, 3];
```

```
arr.splice(1, 1);  
console.log(arr);
```

Output:

[1, 3]

Iterating Arrays

for Loop

```
let arr = [10, 20, 30];  
  
for (let i = 0; i < arr.length; i++) {  
    console.log(arr[i]);  
}
```

for...of Loop

```
let arr = [10, 20, 30];  
  
for (let value of arr) {  
    console.log(value);  
}
```

Advanced Array Methods

map()

Creates new array by transforming elements.

```
let arr = [1, 2, 3];  
  
let result = arr.map(x => x * 2);  
console.log(result);
```

Output:

[2, 4, 6]

filter()

Filters elements based on condition.

```
let arr = [1, 2, 3, 4];  
  
let result = arr.filter(x => x > 2);  
console.log(result);
```

Output:

[3, 4]

reduce()

Reduces array to single value.

```
let arr = [1, 2, 3];

let sum = arr.reduce((acc, curr) => acc + curr, 0);
console.log(sum);
```

Output:

6

Real-World Example

```
let prices = [100, 200, 300];

let total = prices.reduce((sum, price) => sum + price, 0);

console.log(total);
```

Output:

600

Array of Objects

```
let users = [
  { name: "Ali", age: 25 },
  { name: "Sara", age: 30 }
];
```



```
console.log(users[1].name);
```

Output:

Sara

Common Mistakes

1. Accessing index that does not exist

Returns undefined

2. Using wrong loop condition

Can cause infinite loop

3. Confusing slice and splice
4. Forgetting return in map/filter
5. Modifying original array unintentionally

Interview Questions

1. What is an array?
2. Difference between push and pop?
3. Difference between slice and splice?
4. What is map function?
5. What is reduce used for?
6. Difference between for and for...of?

Practice Exercises

1. Create an array of numbers
2. Add and remove elements
3. Find element index
4. Use map to double numbers
5. Use filter to get even numbers

Mini Practice Task

Create array:

- Store 5 numbers
- Find sum using reduce

Challenge Task

Create student array:

- Store objects (name, marks)
- Filter students with marks > 50
- Print names

CHAPTER 12: DESTRUCTURING

Introduction

Destructuring is a feature in JavaScript that allows extracting values from arrays and objects into separate variables. It makes code shorter, cleaner, and easier to read.

Why Destructuring is Important

Without destructuring, accessing values requires multiple lines.

Example without destructuring:

```
let arr = [10, 20, 30];  
  
let a = arr[0];  
let b = arr[1];  
let c = arr[2];
```

With destructuring:

```
let arr = [10, 20, 30];  
  
let [a, b, c] = arr;
```

Array Destructuring

Extract values from arrays.

```
let fruits = ["Apple", "Banana", "Mango"];

let [f1, f2, f3] = fruits;

console.log(f1, f2, f3);
```

Output:

Apple Banana Mango

Skipping Values

```
let numbers = [1, 2, 3, 4];

let [a, , c] = numbers;

console.log(a, c);
```

Output:

1 3

Default Values

```
let arr = [1];

let [a, b = 5] = arr;

console.log(a, b);
```

Output:

1 5

Swapping Variables

```
let x = 10;  
let y = 20;  
  
[x, y] = [y, x];  
  
console.log(x, y);
```

Output:

20 10

Rest Operator in Array Destructuring

```
let arr = [1, 2, 3, 4];  
  
let [a, ...rest] = arr;  
  
console.log(a);  
console.log(rest);
```

Output:

1

[2, 3, 4]

Object Destructuring

Extract values from objects.

```
let user = {  
  name: "Ali",  
  age: 25  
};  
  
let { name, age } = user;  
  
console.log(name, age);
```

Output:

Ali 25

Renaming Variables

```
let user = {  
  name: "Ali",  
  age: 25  
};  
  
let { name: username, age: userAge } = user;  
  
console.log(username, userAge);
```

Output:

Ali 25

Default Values in Objects

```
let user = {  
  name: "Ali"  
};  
  
let { name, age = 20 } = user;  
  
console.log(name, age);
```

Output:

Ali 20

Nested Destructuring

```
let user = {  
  name: "Ali",  
  address: {  
    city: "Delhi"  
  }  
};  
  
let { address: { city } } = user;  
  
console.log(city);
```

Output:

Delhi

Destructuring in Function Parameters

```
function display({ name, age }) {  
    console.log(name, age);  
}  
  
display({ name: "Ali", age: 25 });
```

Output:

Ali 25

Real-World Example

```
let product = {  
    name: "Laptop",  
    price: 50000,  
    category: "Electronics"  
};  
  
let { name, price } = product;  
  
console.log(name + " costs " + price);
```

Output:

Laptop costs 50000

Destructuring Arrays of Objects

```
let users = [  
  { name: "Ali", age: 25 },  
  { name: "Sara", age: 30 }  
];  
  
let [{ name }, { age }] = users;  
  
console.log(name, age);
```

Output:

Ali 30

Common Mistakes

1. Wrong variable names in object destructuring
2. Keys must match
3. Forgetting default values
4. Confusing array and object destructuring
5. Not handling undefined values
6. Incorrect nested destructuring

Interview Questions

1. What is destructuring?
2. Difference between array and object destructuring?
3. How to skip elements in array destructuring?
4. How to rename variables in object destructuring?

5. What is rest operator in destructuring?
6. Can destructuring be used in function parameters?

Practice Exercises

1. Destructure an array of 5 elements
2. Swap two variables using destructuring
3. Extract values from object
4. Use default values in destructuring
5. Use destructuring in function parameter

Mini Practice Task

Create object:

- name
- price
- quantity

Destructure and print values

Challenge Task

Create array of objects:

- Extract names using destructuring
- Print only names

CHAPTER 13: SPREAD AND REST OPERATORS

Introduction

The spread (...) and rest (...) operators are powerful features in JavaScript used to handle arrays, objects, and function parameters.

Although both use the same syntax (...), their purpose is different:

- Spread → Expands values
- Rest → Collects values

Spread Operator

The spread operator expands elements of arrays or objects.

Spread with Arrays

```
let arr1 = [1, 2, 3];  
let arr2 = [...arr1];  
  
console.log(arr2);
```

Output:

[1, 2, 3]

Explanation:

Copies all elements from arr1 into arr2.

Merging Arrays

```
let arr1 = [1, 2];  
let arr2 = [3, 4];  
  
let combined = [...arr1, ...arr2];  
  
console.log(combined);
```

Output:

```
[1, 2, 3, 4]
```

Adding Elements

```
let arr = [2, 3];  
  
let newArr = [1, ...arr, 4];  
  
console.log(newArr);
```

Output:

```
[1, 2, 3, 4]
```

Spread with Objects

```
let obj1 = { name: "Ali" };  
let obj2 = { age: 25 };  
  
let merged = { ...obj1, ...obj2 };  
  
console.log(merged);
```

Output:

```
{ name: "Ali", age: 25 }
```

Overwriting Properties

```
let obj1 = { name: "Ali", age: 20 };  
  
let obj2 = { age: 30 };  
  
let result = { ...obj1, ...obj2 };  
  
console.log(result);
```

Output:

```
{ name: "Ali", age: 30 }
```

Copying Objects

```
let obj1 = { a: 1 };  
let obj2 = { ...obj1 };
```

```
obj2.a = 5;
```

```
console.log(obj1.a);
```

Output:

1

Spread in Function Calls

```
function sum(a, b, c) {  
    return a + b + c;  
}
```

```
let arr = [1, 2, 3];
```

```
console.log(sum(...arr));
```

Output:

6

Rest Operator

The rest operator collects multiple values into a single array.

Rest in Function Parameters

```
function add(...numbers) {  
  let sum = 0;  
  
  for (let num of numbers) {  
    sum += num;  
  }  
  
  return sum;  
}  
  
console.log(add(1, 2, 3, 4));
```

Output:

10

Rest with Array Destructuring

```
let arr = [1, 2, 3, 4];  
  
let [a, ...rest] = arr;  
  
console.log(a);  
console.log(rest);
```

Output:

1

[2, 3, 4]

Rest with Object Destructuring

```
let user = {  
  name: "Ali",  
  age: 25,  
  city: "Delhi"  
};  
  
let { name, ...rest } = user;  
  
console.log(name);  
console.log(rest);
```

Output:

Ali

{ age: 25, city: "Delhi" }

Spread vs Rest

Feature	Spread	Rest
Purpose	Expands values	Collects values
Usage	Arrays/objects	Function parameters
Position	Right side	Left side

Real-World Example

```
let cart1 = ["Laptop", "Phone"];  
let cart2 = ["Tablet"];  
  
let finalCart = [...cart1, ...cart2];
```

```
console.log(finalCart);
```

Output:

```
["Laptop", "Phone", "Tablet"]
```

Function Example with Rest

```
function calculateTotal(...prices) {  
    return prices.reduce((total, price) => total + price,  
    0);  
}
```

```
console.log(calculateTotal(100, 200, 300));
```

Output:

```
600
```

Common Mistakes

1. Confusing spread and rest

Both look same but behave differently

2. Using rest parameter in wrong position

Must be last parameter

3. Expecting deep copy with spread

Spread creates shallow copy

4. Overwriting properties unintentionally
5. Misusing spread in nested objects

Interview Questions

1. What is spread operator?
2. What is rest operator?
3. Difference between spread and rest?
4. How to merge arrays using spread?
5. Can rest be used in function parameters?
6. Does spread create deep copy?

Practice Exercises

1. Copy an array using spread
2. Merge two arrays
3. Create function using rest parameters
4. Extract first element and rest from array
5. Copy object and modify value

Mini Practice Task

Create function:

- Accept multiple numbers using rest
- Return sum

Challenge Task

Create shopping cart:

- Combine multiple arrays using spread
- Calculate total using rest

CHAPTER 14: WHAT IS DOM

Introduction

The DOM (Document Object Model) is a programming interface that represents an HTML document as a tree structure. It allows JavaScript to access, modify, add, and delete elements on a webpage dynamically.

Without the DOM, JavaScript cannot interact with HTML content.

What is DOM

DOM is a structured representation of an HTML page where:

- Each element is treated as an object
- JavaScript can access and modify these objects

Example HTML:

```
<!DOCTYPE html>
<html>
<body>
    <h1>Hello</h1>
</body>
</html>
```

DOM representation:

- Document
 - html
 - body

- h1

DOM Tree Structure

The HTML page is converted into a tree-like structure:

- Document (root)
 - html
 - head
 - body
 - elements (h1, p, div, etc.)

Each node in this structure is called a DOM node.

Types of DOM Nodes

1. Element Nodes → HTML tags (div, p, h1)
2. Text Nodes → Text inside elements
3. Attribute Nodes → Attributes like id, class

How JavaScript Uses DOM

JavaScript can:

- Select elements
- Change content
- Change styles
- Add or remove elements
- Handle user events

Example: Accessing DOM

```
<p id="text">Old Text</p>
```

```
<script>  
document.getElementById("text").innerText = "New Text";  
</script>
```

Explanation:

JavaScript selects the paragraph and changes its text.

Output:

"Old Text" becomes "New Text"

document Object

The document object represents the entire webpage.

Examples:

```
console.log(document);  
console.log(document.title);
```

Accessing Page Information

```
console.log(document.body);  
console.log(document.head);
```

Changing Content

```
document.body.innerHTML = "<h1>Changed</h1>";
```

DOM Manipulation Basics

Selecting Element

```
let element = document.getElementById("text");
```

Changing Text

```
element.innerText = "Updated";
```

Changing HTML

```
element.innerHTML = "<b>Bold Text</b>";
```

Real-World Example

```
<h2 id="heading">Welcome</h2>
```

```
<script>
```

```
let h = document.getElementById("heading");
```

```
h.innerText = "Welcome to JavaScript";
```



```
</script>
```

Output:

Heading text changes dynamically.

DOM vs HTML

Feature	HTML	DOM
Type	Static	Dynamic
Role	Structure	Interface
Editable	No	Yes (via JS)

DOM Loading Process

1. Browser loads HTML
2. Creates DOM tree
3. JavaScript interacts with DOM

Important:

JavaScript should run after DOM is loaded.

Script Placement

Correct placement:

```
<body>
```

```
<h1 id="demo">Hello</h1>
```

```
<script src="script.js"></script>
```

</body>

Reason:

Ensures elements exist before JavaScript runs.

DOMContentLoaded Event

Ensures DOM is fully loaded before execution.

```
document.addEventListener("DOMContentLoaded", function() {  
    console.log("DOM Loaded");  
});
```

Common Mistakes

1. Running JavaScript before DOM loads

Leads to null errors

2. Using wrong selectors

Element not found

3. Confusing innerText and innerHTML
4. Overwriting entire DOM accidentally
5. Not checking console errors

Interview Questions

1. What is DOM?
2. What is DOM tree?
3. Difference between HTML and DOM?
4. What is document object?
5. What are DOM nodes?
6. When does DOM load?

Practice Exercises

1. Create HTML page and access element using JS
2. Change text of heading
3. Replace content using innerHTML
4. Print document title
5. Log document object

Mini Practice Task

Create webpage:

- One heading
- Change text using JavaScript

Challenge Task

Create page:

- Two paragraphs
- Change both using JavaScript

- Use different methods (innerText, innerHTML)

CHAPTER 15: SELECTING ELEMENTS

Introduction

Selecting elements is the first step in DOM manipulation. Before changing content, styles, or handling events, JavaScript must first find and access the HTML elements.

JavaScript provides multiple methods to select elements from the DOM.

Common Methods to Select Elements

- `getElementById`
- `getElementsByClassName`
- `getElementsByTagName`
- `querySelector`
- `querySelectorAll`

`getElementById`

Selects a single element using its id.

```
let element = document.getElementById("heading");  
console.log(element);
```

HTML:

```
<h1 id="heading">Hello</h1>
```

Explanation:

Selects the element with id "heading".

Changing Content

```
let element = document.getElementById("heading");  
element.innerText = "Updated Text";
```

Output:

Text changes on screen.

getElementsByName

Selects all elements with a specific class.

```
let elements = document.getElementsByClassName("box");  
console.log(elements);
```

HTML:

```
<p class="box">One</p>  
<p class="box">Two</p>
```

Explanation:

Returns a collection (array-like).

Accessing Elements

```
console.log(elements[0]);
```

getElementsByTagName

Selects elements by tag name.

```
let items = document.getElementsByTagName("p");  
console.log(items);
```

querySelector

Selects the first matching element using CSS selector.

```
let element = document.querySelector("#heading");
```

Examples:

```
document.querySelector(".box");  
document.querySelector("p");
```

querySelectorAll

Selects all matching elements.

```
let elements = document.querySelectorAll(".box");
console.log(elements);
```

Difference Between `querySelector` and `querySelectorAll`

Method	Returns
<code>querySelector</code>	First element
<code>querySelectorAll</code>	All elements

Looping Through Selected Elements

```
let elements = document.querySelectorAll(".box");

elements.forEach(function(el) {
    console.log(el.innerText);
});
```

Output:

One

Two

Real-World Example

```
<ul>
  <li class="item">Item 1</li>
  <li class="item">Item 2</li>
</ul>

<script>
```



```
let items = document.querySelectorAll(".item");

items.forEach(function(item) {
    item.innerText = "Updated";
});
</script>
```

Output:

All items become "Updated"

Selecting Nested Elements

```
let parent = document.getElementById("container");
let child = parent.querySelector(".box");
```

Selecting by Attribute

```
let input = document.querySelector("input[type='text']");
```

Difference Between HTMLCollection and NodeList

Feature	HTMLCollection	NodeList
Type	Live collection	Static collection
Methods	Limited	forEach available

Converting to Array

```
let elements = document.getElementsByClassName("box");  
  
let arr = Array.from(elements);
```

Common Mistakes

1. Using wrong selector (# vs .)
2. Forgetting that `getElementsByClassName` returns collection
3. Trying to use `forEach` on `HTMLCollection`
4. Not checking if element exists
5. Confusing `querySelector` and `querySelectorAll`

Interview Questions

1. What is `getElementById`?
2. Difference between `querySelector` and `querySelectorAll`?
3. What is `HTMLCollection`?
4. Difference between class and id selection?
5. How to loop through selected elements?
6. Which method uses CSS selectors?

Practice Exercises

1. Select element using id
2. Select elements using class
3. Select all paragraphs
4. Use `querySelector` to select first element

5. Use `querySelectorAll` and loop through elements

Mini Practice Task

Create webpage:

- 3 paragraphs
- Change all text using `querySelectorAll`

Challenge Task

Create list:

- Select all items
- Change only even index items

CHAPTER 16: EVENT HANDLING

Introduction

Events are actions that occur in the browser, such as a user clicking a button, typing in an input field, or submitting a form. Event handling is the process of detecting these actions and executing code in response.

JavaScript allows you to attach event listeners to elements and define what should happen when an event occurs.

What is an Event

An event is any interaction or occurrence in the browser.

Examples:

- click
- input
- submit
- mouseover
- keydown

Event Handling Methods

- Inline event handling
- Using event properties
- Using `addEventListener` (recommended)

Inline Event Handling

```
<button onclick="showMessage()">Click</button>
```

```
<script>
function showMessage() {
    console.log("Button clicked");
}
</script>
```

Explanation:

Event is written directly inside HTML.

Using Event Property

```
<button id="btn">Click</button>
```

```
<script>
let btn = document.getElementById("btn");

btn.onclick = function() {
    console.log("Clicked");
};
</script>
```

addEventListener (Best Practice)

```
<button id="btn">Click</button>
```

```
<script>
```

```
let btn = document.getElementById("btn");

btn.addEventListener("click", function() {
    console.log("Button clicked");
});
</script>
```

Advantages:

- Multiple events can be attached
- Cleaner code
- Better control

Common Event Types

Event	Description
click	When element is clicked
input	When user types
change	When value changes
submit	When form is submitted
mouseover	Mouse enters element
keydown	Key is pressed

Example: Input Event

```
<input type="text" id="name">

<script>
let input = document.getElementById("name");

input.addEventListener("input", function() {
    console.log(input.value);
});
```

```
</script>
```

Explanation:

Prints value as user types.

Event Object

When an event occurs, an event object is automatically passed.

```
document.getElementById("btn").addEventListener("click",  
function(event) {  
    console.log(event);  
});
```

Accessing Event Properties

```
document.getElementById("btn").addEventListener("click",  
function(event) {  
    console.log(event.target);  
});
```

Explanation:

`event.target` gives the element that triggered the event.

Prevent Default Behavior

Used to stop default browser behavior.

```
<form id="form">
  <button type="submit">Submit</button>
</form>

<script>
document.getElementById("form").addEventListener("submit",
function(event) {
  event.preventDefault();
  console.log("Form submission prevented");
});
</script>
```

Event Bubbling

Events move from child to parent.

```
<div id="parent">
  <button id="child">Click</button>
</div>

<script>
document.getElementById("parent").addEventListener("click",
function() {
  console.log("Parent clicked");
});

document.getElementById("child").addEventListener("click",
function() {
  console.log("Child clicked");
});
</script>
```


Output:

Child clicked

Parent clicked

Event Capturing

Events move from parent to child.

```
element.addEventListener("click", handler, true);
```

Event Delegation

Handling events on parent instead of individual elements.

```
<ul id="list">
  <li>Item 1</li>
  <li>Item 2</li>
</ul>

<script>
document.getElementById("list").addEventListener("click",
function(e) {
  console.log(e.target.innerText);
});
</script>
```

Explanation:

Handles clicks for all list items.

Real-World Example

```
<input type="text" id="username">
<button id="btn">Submit</button>
<p id="output"></p>

<script>
let btn = document.getElementById("btn");

btn.addEventListener("click", function() {
    let value = document.getElementById("username").value;
    document.getElementById("output").innerText = value;
});
</script>
```

Output:

Displays input value when button is clicked.

Common Mistakes

1. Forgetting to attach event listener
2. Using wrong event type
3. Not using preventDefault in forms
4. Confusing event.target and this
5. Not understanding event bubbling

Interview Questions

1. What is an event in JavaScript?

2. What is addEventListener?
3. Difference between onclick and addEventListener?
4. What is event object?
5. What is event bubbling?
6. What is event delegation?

Practice Exercises

1. Create button click event
2. Display input value on typing
3. Handle form submit event
4. Use event object
5. Implement event delegation

Mini Practice Task

Create webpage:

- Input field
- Button
- Display typed text when button clicked

Challenge Task

Create interactive list:

- Add items
- Click item to highlight
- Use event delegation

CHAPTER 17: DOM PROJECT — TODO APP

Introduction

This chapter builds a complete Todo App using JavaScript and DOM manipulation. The project covers:

- Selecting elements
- Handling events
- Creating and deleting elements
- Updating UI dynamically
- Storing data

This project is commonly asked in interviews and demonstrates practical JavaScript skills.

Problem Statement

Create a Todo App where a user can:

- Add tasks
- Delete tasks
- Mark tasks as completed

Project Structure

```
project/  
|
```

- |— index.html
- |— style.css
- |— script.js

Step 1: HTML Structure

```
<!DOCTYPE html>
<html>
<head>
  <title>Todo App</title>
</head>
<body>

<h2>Todo App</h2>

<input type="text" id="taskInput" placeholder="Enter task">
<button id="addBtn">Add Task</button>

<ul id="taskList"></ul>

<script src="script.js"></script>

</body>
</html>
```

Step 2: Basic JavaScript Setup

```
let input = document.getElementById("taskInput");
let button = document.getElementById("addBtn");
```

```
let list = document.getElementById("taskList");
```

Step 3: Add Task Function

```
button.addEventListener("click", function() {  
    let task = input.value;  
  
    if (task === "") {  
        alert("Enter a task");  
        return;  
    }  
  
    let li = document.createElement("li");  
    li.innerText = task;  
  
    list.appendChild(li);  
  
    input.value = "";  
});
```

Explanation

- Get value from input
- Create new list item
- Add text to it
- Append to list
- Clear input field

Output

When user types "Study JavaScript" and clicks Add:

- A new list item appears

Step 4: Delete Task

```
button.addEventListener("click", function() {
    let task = input.value;

    if (task === "") return;

    let li = document.createElement("li");

    li.innerText = task;

    let deleteBtn = document.createElement("button");
    deleteBtn.innerText = "Delete";

    deleteBtn.addEventListener("click", function() {
        li.remove();
    });

    li.appendChild(deleteBtn);
    list.appendChild(li);

    input.value = "";
});
```


Explanation

- Each task has a delete button
- Clicking it removes the task

Step 5: Mark Task as Completed

```
li.addEventListener("click", function() {  
    li.style.textDecoration = "line-through";  
});
```

Combined Version

```
let input = document.getElementById("taskInput");  
let button = document.getElementById("addBtn");  
let list = document.getElementById("taskList");  
  
button.addEventListener("click", function() {  
    let task = input.value;  
  
    if (task === "") return;  
  
    let li = document.createElement("li");  
    li.innerText = task;  
  
    li.addEventListener("click", function() {  
        li.style.textDecoration = "line-through";  
    });  
  
    let deleteBtn = document.createElement("button");  
    deleteBtn.innerText = "Delete";
```

```
deleteBtn.addEventListener("click", function() {  
    li.remove();  
});  
  
li.appendChild(deleteBtn);  
list.appendChild(li);  
  
input.value = "";  
});
```

Step 6: Add Basic Styling (Optional)

```
body {  
    font-family: Arial;  
}  
  
li {  
    margin: 5px 0;  
}  
  
button {  
    margin-left: 10px;  
}
```

Real-World Improvements

- Add local storage
- Add edit feature
- Add filter (completed/pending)

- Add timestamps

Common Mistakes

1. Not clearing input field
2. Adding empty tasks
3. Not attaching delete event correctly
4. Overwriting event listeners
5. Forgetting to append elements

Interview Questions

1. How does DOM manipulation work in this project?
2. How do you create elements dynamically?
3. How do you remove elements?
4. How can you store tasks permanently?
5. What improvements can be added?

Practice Exercises

1. Add alert for empty input
2. Add checkbox instead of click for completion
3. Add edit functionality
4. Count total tasks
5. Add clear all button

Mini Practice Task

Modify app:

- Add "Clear All" button
- Remove all tasks when clicked

Challenge Task

Enhance Todo App:

- Store tasks in local storage
- Load tasks when page reloads

CHAPTER 18: CALLBACKS

Introduction

Callbacks are functions passed as arguments to other functions and executed later. They are a core concept in JavaScript, especially for handling asynchronous operations like events, timers, and API calls.

What is a Callback Function

A callback is a function given to another function to be executed after a certain task is completed.

Example:

```
function greet(name, callback) {  
    console.log("Hello " + name);  
    callback();  
}
```

```
function sayBye() {  
    console.log("Goodbye");  
}
```

```
greet("Ali", sayBye);
```

Output:

Hello Ali

Goodbye

Why Callbacks are Used

- To handle asynchronous operations
- To control execution order
- To reuse logic

Synchronous vs Asynchronous

Synchronous Code

Executes line by line.

```
console.log("Start");  
console.log("End");
```

Output:

Start

End

Asynchronous Code

Does not block execution.

```
console.log("Start");  
  
setTimeout(function() {  
    console.log("Delayed");  
}, 2000);
```

```
console.log("End");
```

Output:

Start

End

Delayed

Callback in Asynchronous Code

```
function fetchData(callback) {  
    setTimeout(function() {  
        console.log("Data fetched");  
        callback();  
    }, 2000);  
}
```

```
fetchData(function() {  
    console.log("Processing data");  
});
```

Output:

Data fetched

Processing data

Callback Example: Array Method

```
let numbers = [1, 2, 3];

numbers.forEach(function(num) {
    console.log(num);
});
```

Output:

1
2
3

Explanation:

The function passed to `forEach` is a callback.

Callback Example: Event Handling

```
document.getElementById("btn").addEventListener("click",
function() {
    console.log("Button clicked");
});
```

Explanation:

Function runs when button is clicked.

Callback Hell

Nested callbacks make code difficult to read and maintain.

Example:

```
setTimeout(function() {  
    console.log("Step 1");  
  
    setTimeout(function() {  
        console.log("Step 2");  
  
        setTimeout(function() {  
            console.log("Step 3");  
        }, 1000);  
    }, 1000);  
}, 1000);
```

Problems with Callback Hell

- Hard to read
- Difficult to debug
- Poor maintainability

Real-World Example

```
function login(user, callback) {  
    console.log("User logged in:", user);  
  
    callback();  
}
```

```
}  
  
function loadDashboard() {  
    console.log("Dashboard loaded");  
}  
  
login("Ali", loadDashboard);
```

Output:

User logged in: Ali

Dashboard loaded

Error Handling in Callbacks

```
function processData(callback) {  
    let success = true;  
  
    if (success) {  
        callback(null, "Data processed");  
    } else {  
        callback("Error occurred", null);  
    }  
}  
  
processData(function(error, result) {  
    if (error) {  
        console.log(error);  
    } else {  
        console.log(result);  
    }  
})
```

```
});
```

Common Mistakes

1. Forgetting to call callback
2. Calling callback multiple times
3. Deep nesting of callbacks
4. Not handling errors
5. Losing track of execution flow

Interview Questions

1. What is a callback function?
2. Why are callbacks used?
3. Difference between synchronous and asynchronous code?
4. What is callback hell?
5. How to handle errors in callbacks?
6. Give real-world example of callback

Practice Exercises

1. Create function that accepts callback
2. Use callback with setTimeout
3. Use callback in array method
4. Create login system using callback
5. Write nested callback example

Mini Practice Task

Create function:

- Accept name and callback
- Print name
- Then call callback

Challenge Task

Create flow:

- Step 1 → Load user
- Step 2 → Load orders
- Step 3 → Show result

Use nested callbacks

CHAPTER 19: PROMISES

Introduction

Promises are used to handle asynchronous operations in a cleaner and more structured way than callbacks. They help avoid deeply nested code (callback hell) and make code easier to read and maintain.

What is a Promise

A Promise is an object that represents the eventual result (success or failure) of an asynchronous operation.

A promise has three states:

- Pending → Initial state
- Fulfilled → Operation successful
- Rejected → Operation failed

Creating a Promise

```
let promise = new Promise(function(resolve, reject) {  
    let success = true;  
  
    if (success) {  
        resolve("Operation successful");  
    } else {  
        reject("Operation failed");  
    }  
})
```

```
});
```

Consuming a Promise

```
promise
  .then(function(result) {
    console.log(result);
  })
  .catch(function(error) {
    console.log(error);
  });
```

Output:

Operation successful

then() Method

Used to handle successful result.

```
let p = new Promise(function(resolve) {
  resolve("Success");
});

p.then(function(data) {
  console.log(data);
});
```

catch() Method

Used to handle errors.

```
let p = new Promise(function(resolve, reject) {  
    reject("Error occurred");  
});  
  
p.catch(function(error) {  
    console.log(error);  
});
```

finally() Method

Runs regardless of success or failure.

```
p.finally(function() {  
    console.log("Completed");  
});
```

Promise Chaining

```
let p = new Promise(function(resolve) {  
    resolve(5);  
});  
  
p.then(function(num) {  
    return num * 2;  
})  
  .then(function(result) {  
    console.log(result);  
});
```

```
});
```

Output:

10

Example: Asynchronous Task

```
function fetchData() {  
    return new Promise(function(resolve) {  
        setTimeout(function() {  
            resolve("Data fetched");  
        }, 2000);  
    });  
}  
  
fetchData().then(function(data) {  
    console.log(data);  
});
```

Output:

Data fetched

Handling Errors in Promise Chain

```
let p = new Promise(function(resolve, reject) {  
    reject("Something went wrong");  
});  
  
p.then(function(data) {
```



```
        console.log(data);
    })
    .catch(function(error) {
        console.log(error);
    });
```

Multiple Promises

Promise.all()

Runs multiple promises together.

```
let p1 = Promise.resolve(10);
let p2 = Promise.resolve(20);
```

```
Promise.all([p1, p2])
    .then(function(values) {
        console.log(values);
    });
```

Output:

```
[10, 20]
```

Promise.race()

Returns first completed promise.

```
let p1 = new Promise(resolve => setTimeout(() =>
    resolve("First"), 1000));
let p2 = new Promise(resolve => setTimeout(() =>
    resolve("Second"), 2000));
```

```
Promise.race([p1, p2])
  .then(function(result) {
    console.log(result);
  });
```

Output:

First

Real-World Example

```
function loginUser() {
  return new Promise(function(resolve) {
    setTimeout(function() {
      resolve("User logged in");
    }, 1000);
  });
}

function getData() {
  return new Promise(function(resolve) {
    setTimeout(function() {
      resolve("Data loaded");
    }, 1000);
  });
}

loginUser()
  .then(function(msg) {
    console.log(msg);
    return getData();
  })
```

```
.then(function(data) {  
    console.log(data);  
});
```

Output:

User logged in

Data loaded

Advantages of Promises

- Avoid callback hell
- Better readability
- Easier error handling
- Supports chaining

Common Mistakes

1. Not returning promise in chain
2. Forgetting catch for error handling
3. Mixing callbacks and promises
4. Not understanding promise states
5. Writing unnecessary nested promises

Interview Questions

1. What is a Promise?
2. What are its states?
3. Difference between then and catch?

4. What is promise chaining?
5. What is Promise.all()?
6. What is Promise.race()?

Practice Exercises

1. Create a simple promise
2. Resolve and reject promise
3. Chain multiple then methods
4. Handle errors using catch
5. Use Promise.all

Mini Practice Task

Create promise:

- Wait 2 seconds
- Return "Task completed"

Challenge Task

Create flow using promises:

- Login
- Fetch data
- Display result

CHAPTER 20: ASYNC / AWAIT

Introduction

Async/Await is a modern way to handle asynchronous operations in JavaScript. It is built on top of Promises and makes asynchronous code look and behave like synchronous code, improving readability and maintainability.

Why Async/Await

Problems with Promises:

- Multiple `.then()` chains reduce readability
- Nested logic becomes hard to follow

Async/Await solves this by writing cleaner, step-by-step code.

async Keyword

The `async` keyword is used to declare a function that returns a Promise.

```
async function greet() {  
    return "Hello";  
}  
  
greet().then(function(result) {  
    console.log(result);  
});
```

Output:

Hello

await Keyword

The `await` keyword pauses execution until a Promise is resolved.

It can only be used inside an async function.

```
function fetchData() {  
    return new Promise(function(resolve) {  
        setTimeout(function() {  
            resolve("Data fetched");  
        }, 2000);  
    });  
}  
  
async function getData() {  
    let result = await fetchData();  
    console.log(result);  
}  
  
getData();
```

Output:

Data fetched

Example Without Async/Await (Using Promises)

```
fetchData()  
  .then(function(data) {  
    console.log(data);  
  });
```

Same Example With Async/Await

```
async function getData() {  
  let data = await fetchData();  
  console.log(data);  
}
```

Handling Multiple Await

```
function step1() {  
  return Promise.resolve("Step 1 done");  
}
```

```
function step2() {  
  return Promise.resolve("Step 2 done");  
}
```

```
async function process() {  
  let res1 = await step1();  
  console.log(res1);  
  
  let res2 = await step2();  
  console.log(res2);  
}
```

```
}  
  
process();
```

Output:

Step 1 done

Step 2 done

Error Handling with try...catch

```
async function test() {  
  try {  
    let result = await Promise.reject("Error occurred");  
    console.log(result);  
  } catch (error) {  
    console.log(error);  
  }  
}  
  
test();
```

Output:

Error occurred

Real-World Example

```
function login() {  
  return new Promise(resolve => {
```



```

        setTimeout(() => resolve("User logged in"), 1000);
    });
}

function getProfile() {
    return new Promise(resolve => {
        setTimeout(() => resolve("Profile loaded"), 1000);
    });
}

async function startApp() {
    let user = await login();
    console.log(user);

    let profile = await getProfile();
    console.log(profile);
}

startApp();

```

Output:

User logged in

Profile loaded

Parallel Execution (Important)

```

async function run() {
    let p1 = fetchData();
    let p2 = fetchData();

    let res1 = await p1;

```

```
    let res2 = await p2;

    console.log(res1, res2);
}
```

Using Promise.all with Async/Await

```
async function run() {
    let results = await Promise.all([fetchData(),
    fetchData()]);
    console.log(results);
}
```

Rules of Async/Await

- await only works inside async functions
- async functions always return a Promise
- use try...catch for error handling

Common Mistakes

1. Using await outside async function
2. Forgetting try...catch
3. Writing sequential code when parallel needed
4. Not understanding async returns promise
5. Mixing callbacks, promises, and async incorrectly

Interview Questions

1. What is async/await?
2. Difference between Promise and async/await?
3. Can we use await outside async?
4. How to handle errors in async/await?
5. Does async function return a promise?
6. How to run multiple async tasks in parallel?

Practice Exercises

1. Convert promise code to async/await
2. Create async function with delay
3. Handle error using try...catch
4. Use multiple await calls
5. Use Promise.all with async

Mini Practice Task

Create async function:

- Wait 2 seconds
- Print "Completed"

Challenge Task

Create system:

- Login
- Fetch orders

- Display result

Use `async/await`

CHAPTER 22: PROTOTYPES AND INHERITANCE

Introduction

JavaScript uses prototypes to implement inheritance. Every object in JavaScript has a hidden property called `[[Prototype]]`, which links it to another object. This mechanism is called the prototype chain.

Understanding prototypes is important for:

- Object reuse
- Memory efficiency
- Understanding how JavaScript works internally

What is Prototype

A prototype is an object from which other objects inherit properties and methods.

Every JavaScript object has a prototype.

Example: Basic Object

```
let obj = {  
  name: "Ali"  
};  
  
console.log(obj);
```

Even though we only define `name`, the object has many built-in methods like `toString()`. These come from its prototype.

Prototype Chain

If a property is not found in the object, JavaScript looks in its prototype.

If not found there, it continues up the chain.

Example: Prototype Chain

```
let arr = [1, 2, 3];  
  
console.log(arr.toString());
```

Explanation:

`toString()` is not defined in array, but it exists in its prototype.

proto (Accessing Prototype)

```
let obj = {};  
console.log(obj.__proto__);
```

Note:

`__proto__` is used for learning, not recommended in production.

Using Object.create()

Creates a new object with specified prototype.

```
let parent = {  
  greet: function() {  
    console.log("Hello");  
  }  
};  
  
let child = Object.create(parent);  
  
child.greet();
```

Output:

Hello

Constructor Functions and Prototype

```
function Person(name) {  
  this.name = name;  
}  
  
Person.prototype.sayHello = function() {  
  console.log("Hello " + this.name);  
};  
  
let p1 = new Person("Ali");  
p1.sayHello();
```

Output:

Hello Ali

Why Use Prototype

If methods are defined inside constructor:

```
function Person(name) {  
    this.name = name;  
  
    this.sayHello = function() {  
        console.log("Hello " + this.name);  
    };  
}
```

Each object gets its own copy → memory waste

Using prototype → shared method

Inheritance Using Prototype

```
let animal = {  
    eat: function() {  
        console.log("Eating");  
    }  
};  
  
let dog = Object.create(animal);  
  
dog.bark = function() {  
    console.log("Barking");  
};
```



```
dog.eat();  
dog.bark();
```

Output:

Eating

Barking

Prototype Chain Example

```
let grandparent = {  
  greet: function() {  
    console.log("Hello from grandparent");  
  }  
};
```

```
let parent = Object.create(grandparent);  
let child = Object.create(parent);
```

```
child.greet();
```

Output:

Hello from grandparent

hasOwnProperty()

Checks if property belongs to object itself.

```
let obj = { name: "Ali" };
```

```
console.log(obj.hasOwnProperty("name")); // true
console.log(obj.hasOwnProperty("toString")); // false
```

Real-World Example

```
function Product(name, price) {
  this.name = name;
  this.price = price;
}

Product.prototype.getPrice = function() {
  return this.price;
};

let p = new Product("Laptop", 50000);

console.log(p.getPrice());
```

Output:

50000

Difference: Prototype vs Class

Feature	Prototype	Class (ES6)
Style	Functional	Syntactic sugar
Usage	Low-level control	Easier syntax

Common Mistakes

1. Defining methods inside constructor instead of prototype
2. Misunderstanding prototype chain
3. Using **proto** in production
4. Confusing object copy with inheritance
5. Not using `hasOwnProperty`

Interview Questions

1. What is prototype in JavaScript?
2. What is prototype chain?
3. Difference between **proto** and prototype?
4. Why use prototype instead of constructor methods?
5. What is inheritance in JavaScript?
6. What is `Object.create()`?

Practice Exercises

1. Create object and access prototype
2. Create constructor and add method using prototype
3. Implement inheritance using `Object.create`
4. Use `hasOwnProperty`
5. Trace prototype chain

Mini Practice Task

Create constructor:

- User (name, age)

- Add method using prototype
- Print details

Challenge Task

Create system:

- Animal → eat()
- Dog → bark()
- Use prototype inheritance

CHAPTER 23: CLASSES (ES6)

Introduction

Classes in JavaScript provide a cleaner and more structured way to create objects and implement inheritance. They are built on top of prototypes but offer a syntax similar to other programming languages.

Classes make code easier to read, maintain, and organize.

What is a Class

A class is a blueprint for creating objects.

Example:

```
class Person {  
    constructor(name, age) {  
        this.name = name;  
        this.age = age;  
    }  
}  
  
let p1 = new Person("Ali", 25);  
console.log(p1.name);
```

Output:

Ali

Constructor Method

The constructor is a special method used to initialize object properties.

```
class Car {  
    constructor(brand) {  
        this.brand = brand;  
    }  
}  
  
let car1 = new Car("BMW");  
console.log(car1.brand);
```

Adding Methods

Methods are functions inside a class.

```
class Person {  
    constructor(name) {  
        this.name = name;  
    }  
  
    greet() {  
        console.log("Hello " + this.name);  
    }  
}  
  
let user = new Person("Ali");  
user.greet();
```

Output:

Hello Ali

Class Expression

Classes can also be stored in variables.

```
let Person = class {  
    constructor(name) {  
        this.name = name;  
    }  
};  
  
let p = new Person("Ali");
```

Inheritance using extends

Classes can inherit properties and methods from another class.

```
class Animal {  
    eat() {  
        console.log("Eating");  
    }  
}  
  
class Dog extends Animal {  
    bark() {  
        console.log("Barking");  
    }  
}  
  
let d = new Dog();  
d.eat();
```

```
d.bark();
```

Output:

Eating

Barking

super Keyword

Used to call parent class constructor.

```
class Animal {
    constructor(name) {
        this.name = name;
    }
}

class Dog extends Animal {
    constructor(name, breed) {
        super(name);
        this.breed = breed;
    }
}

let d = new Dog("Tommy", "Labrador");
console.log(d.name, d.breed);
```

Output:

Tommy Labrador

Method Overriding

Child class can override parent methods.

```
class Animal {  
    speak() {  
        console.log("Animal sound");  
    }  
}  
  
class Dog extends Animal {  
    speak() {  
        console.log("Bark");  
    }  
}  
  
let d = new Dog();  
d.speak();
```

Output:

Bark

Static Methods

Belong to class, not objects.

```
class MathUtil {  
    static add(a, b) {  
        return a + b;  
    }  
}
```

```
console.log(MathUtil.add(2, 3));
```

Output:

5

Getters and Setters

Used to control access to properties.

```
class Person {  
    constructor(name) {  
        this._name = name;  
    }  
  
    get name() {  
        return this._name;  
    }  
  
    set name(value) {  
        this._name = value;  
    }  
}
```

```
let p = new Person("Ali");  
console.log(p.name);
```

```
p.name = "Sara";  
console.log(p.name);
```

Output:

Ali

Sara

Real-World Example

```
class Product {
  constructor(name, price) {
    this.name = name;
    this.price = price;
  }

  getPrice() {
    return this.price;
  }
}

class DiscountedProduct extends Product {
  constructor(name, price, discount) {
    super(name, price);
    this.discount = discount;
  }

  finalPrice() {
    return this.price - (this.price * this.discount /
100);
  }
}

let item = new DiscountedProduct("Laptop", 50000, 10);
console.log(item.finalPrice());
```

Output:

45000

Difference: Class vs Function Constructor

Feature	Class	Function Constructor
Syntax	Cleaner	Older style
Readability	High	Medium
Inheritance	Easy (extends)	Complex

Common Mistakes

1. Forgetting to use new keyword
2. Not calling super() in child constructor
3. Confusing static methods with instance methods
4. Using class without understanding prototype
5. Naming conflicts with getters/setters

Interview Questions

1. What is a class in JavaScript?
2. Difference between class and constructor function?
3. What is inheritance in classes?
4. What is super keyword?
5. What are static methods?
6. What are getters and setters?

Practice Exercises

1. Create a class for Student
2. Add methods inside class
3. Create child class using extends
4. Override method

5. Use static method

Mini Practice Task

Create class:

- User (name, age)
- Method to display details

Challenge Task

Create system:

- Vehicle → speed
- Car → brand
- Add method to calculate speed

CHAPTER 24: MODULES

Introduction

Modules allow you to split JavaScript code into separate files and reuse them. This improves code organization, maintainability, and scalability.

In modern JavaScript, modules use `export` and `import`.

Why Use Modules

- Organize code into smaller files
- Avoid global scope pollution
- Reuse code
- Improve readability

Basic Concept

Each JavaScript file is treated as a module.

To use code from one file in another:

- Export from one file
- Import into another

Exporting from a Module

Named Export

```
// math.js

export function add(a, b) {
  return a + b;
}

export function subtract(a, b) {
  return a - b;
}
```

Importing Named Exports

```
// main.js

import { add, subtract } from './math.js';

console.log(add(5, 3));
console.log(subtract(5, 3));
```

Output:

8

2

Default Export

A file can have one default export.

```
// greet.js

export default function greet() {
  console.log("Hello");
}
```

Import Default Export

```
// main.js

import greet from './greet.js';

greet();
```

Output:

Hello

Exporting Variables

```
export const pi = 3.14;
```


Import All

```
import * as math from './math.js';  
  
console.log(math.add(2, 3));
```

Renaming Imports

```
import { add as sum } from './math.js';  
  
console.log(sum(2, 3));
```

Module in HTML

To use modules in browser:

```
<script type="module" src="main.js"></script>
```

Important Rules

- Use type="module" in script
- Use relative path (./)
- Modules run in strict mode

Example Project Structure

```
project/
|
|— index.html
|— main.js
|— math.js
|— utils.js
```

Real-World Example

math.js

```
export function calculateTotal(price, quantity) {
  return price * quantity;
}
```

main.js

```
import { calculateTotal } from './math.js';

console.log(calculateTotal(100, 2));
```

Output:

200

Benefits of Modules

- Cleaner code
- Better debugging
- Reusability
- Separation of concerns

Common Mistakes

1. Forgetting `type="module"`
2. Wrong file path
3. Using default and named exports incorrectly
4. Not using `.js` extension
5. Trying to use modules without server (in some cases)

Interview Questions

1. What are JavaScript modules?
2. Difference between default and named export?
3. How to import all functions?
4. Why use modules?
5. What is `type="module"`?

Practice Exercises

1. Create file and export function
2. Import function in another file
3. Use default export
4. Rename import
5. Use `import *`

Mini Practice Task

Create:

- `math.js` → add function
- `main.js` → import and use

Challenge Task

Create project:

- `utils.js` → helper functions
- `api.js` → API logic
- `main.js` → combine all

CHAPTER 25: ERROR HANDLING

Introduction

Error handling is the process of detecting, managing, and responding to errors in a program. Proper error handling prevents applications from crashing and helps developers debug issues effectively.

JavaScript provides built-in mechanisms to handle errors safely.

Types of Errors in JavaScript

1. Syntax Errors
2. Runtime Errors
3. Logical Errors

Syntax Errors

Errors in code structure.

```
console.log("Hello"
```

Explanation:

Missing closing parenthesis causes error.

Runtime Errors

Errors during execution.

```
let x = y + 5;
```

Explanation:

Variable `y` is not defined.

Logical Errors

Code runs but produces incorrect output.

```
let sum = 10 + "5";  
console.log(sum);
```

Output:

"105" instead of 15

try...catch Statement

Used to handle runtime errors.

```
try {  
    let result = x + 5;  
} catch (error) {  
    console.log("Error occurred");  
}
```

Example with Error Message

```
try {  
    let result = x + 5;  
} catch (error) {  
    console.log(error.message);  
}
```

finally Block

Executes regardless of error.

```
try {  
    console.log("Try block");  
} catch (error) {  
    console.log("Catch block");  
} finally {  
    console.log("Finally block");  
}
```

Output:

Try block

Finally block

Throwing Custom Errors

```
function checkAge(age) {  
    if (age < 18) {  
        throw new Error("Not allowed");  
    }  
    return "Access granted";  
}  
  
try {  
    console.log(checkAge(15));  
} catch (error) {  
    console.log(error.message);  
}
```

Error Object

Provides details about error.

Properties:

- name
- message
- stack

```
try {  
    let x = y;  
} catch (error) {  
    console.log(error.name);  
    console.log(error.message);  
}
```


Handling Errors in Async Code

```
async function fetchData() {  
  try {  
    let response = await fetch("wrong-url");  
    let data = await response.json();  
    console.log(data);  
  } catch (error) {  
    console.log("Error:", error.message);  
  }  
}
```

Common Error Types

- ReferenceError
- TypeError
- SyntaxError

Example:

```
let x = null;  
console.log(x.toUpperCase());
```

Output:

TypeError

Debugging Techniques

Using console

```
console.log("Debugging");
```

Using Breakpoints

- Open DevTools
- Go to Sources
- Add breakpoint

Using try-catch for debugging

Real-World Example

```
function withdraw(balance, amount) {  
  if (amount > balance) {  
    throw new Error("Insufficient balance");  
  }  
  return balance - amount;  
}  
  
try {  
  let result = withdraw(1000, 1500);  
  console.log(result);  
} catch (error) {  
  console.log(error.message);  
}
```

Output:

Insufficient balance

Best Practices

- Always handle possible errors
- Use meaningful error messages
- Avoid silent failures
- Use try-catch for risky code
- Log errors for debugging

Common Mistakes

1. Ignoring errors
2. Using try-catch for normal flow
3. Not throwing meaningful errors
4. Forgetting finally block when needed
5. Not handling async errors

Interview Questions

1. What is error handling?
2. Types of errors in JavaScript?
3. What is try-catch?
4. What is finally block?
5. How to throw custom error?
6. How to handle async errors?

Practice Exercises

1. Write try-catch example
2. Throw custom error
3. Handle runtime error
4. Print error message
5. Handle async error

Mini Practice Task

Create function:

- Check if number is negative
- Throw error if true

Challenge Task

Create banking system:

- Deposit
- Withdraw
- Handle insufficient balance using errors

CHAPTER 26: WORKING WITH APIs

Introduction

Working with APIs is a core skill in modern web development. APIs allow your application to communicate with external services to fetch or send data.

In real-world applications, APIs are used for:

- Authentication (login/signup)
- Fetching data (users, products, weather)
- Sending data (forms, orders)

What is an API

API (Application Programming Interface) allows two systems to communicate.

Example:

- A frontend app requests data
- Server responds with data

What is a REST API

REST API is the most common type of API.

It uses HTTP methods:

Method	Purpose
GET	Fetch data
POST	Create data
PUT	Update data

DELETE Delete data

API Request Flow

1. Client sends request
2. Server processes request
3. Server sends response
4. Client uses data

Example: GET Request

```
async function getPosts() {  
    let response = await  
fetch("https://jsonplaceholder.typicode.com/posts");  
    let data = await response.json();  
    console.log(data);  
}  
  
getPosts();
```

Understanding JSON

JSON (JavaScript Object Notation) is the format used to transfer data.

Example:

```
{  
  "name": "Ali",  
  "age": 25  
}
```

Displaying API Data

```
<ul id="list"></ul>

<script>
async function loadUsers() {
  let response = await
fetch("https://jsonplaceholder.typicode.com/users");
  let users = await response.json();

  let list = document.getElementById("list");

  users.forEach(user => {
    let li = document.createElement("li");
    li.innerText = user.name;
    list.appendChild(li);
  });
}

loadUsers();
</script>
```

POST Request (Send Data)

```
async function addUser() {
  let response = await
fetch("https://jsonplaceholder.typicode.com/users", {
  method: "POST",
  headers: {
    "Content-Type": "application/json"
  },

```

```
        body: JSON.stringify({
            name: "Ali",
            age: 25
        })
    });

    let data = await response.json();
    console.log(data);
}
```

PUT Request (Update Data)

```
fetch("https://jsonplaceholder.typicode.com/posts/1", {
    method: "PUT",
    body: JSON.stringify({
        title: "Updated Title"
    }),
    headers: {
        "Content-Type": "application/json"
    }
});
```

DELETE Request

```
fetch("https://jsonplaceholder.typicode.com/posts/1", {
    method: "DELETE"
});
```


Handling API Errors

```
async function fetchData() {
  try {
    let response = await fetch("wrong-url");

    if (!response.ok) {
      throw new Error("API error");
    }

    let data = await response.json();
    console.log(data);

  } catch (error) {
    console.log(error.message);
  }
}
```

Headers in API Requests

```
fetch("url", {
  headers: {
    "Authorization": "Bearer token"
  }
});
```

Real-World Example: Weather App

```
async function getWeather() {
  let response = await
```

```
fetch("https://api.example.com/weather?city=Delhi");
  let data = await response.json();

  console.log("Temperature:", data.temperature);
}
```

API Best Practices

- Always check response.ok
- Use async/await for readability
- Handle errors properly
- Use secure APIs (HTTPS)
- Avoid exposing sensitive data

Common Mistakes

1. Forgetting JSON.stringify in POST
2. Not checking response status
3. Wrong API endpoint
4. Ignoring error handling
5. Not setting headers

Interview Questions

1. What is an API?
2. What is REST API?
3. Difference between GET and POST?
4. What is JSON?
5. How to handle API errors?

6. What are HTTP methods?

Practice Exercises

1. Fetch data from API
2. Display data in webpage
3. Send POST request
4. Update data using PUT
5. Delete data using DELETE

Mini Practice Task

Create program:

- Fetch users
- Display names in list

Challenge Task

Create mini dashboard:

- Fetch products
- Display in cards
- Show name, price, category

CHAPTER 27: FORM HANDLING & VALIDATION

Introduction

Forms are used to collect user input such as name, email, password, etc. Form handling involves capturing this input and processing it using JavaScript. Validation ensures that the data entered by the user is correct before submission.

Basic Form Structure

```
<form id="myForm">
  <input type="text" id="name" placeholder="Enter name">
  <input type="email" id="email" placeholder="Enter
email">
  <button type="submit">Submit</button>
</form>
```

Handling Form Submission

```
let form = document.getElementById("myForm");

form.addEventListener("submit", function(event) {
  event.preventDefault();
  console.log("Form submitted");
});
```

Explanation:

`event.preventDefault()` stops the page from reloading.

Getting Input Values

```
form.addEventListener("submit", function(event) {  
    event.preventDefault();  
  
    let name = document.getElementById("name").value;  
    let email = document.getElementById("email").value;  
  
    console.log(name, email);  
});
```

Basic Validation

```
form.addEventListener("submit", function(event) {  
    event.preventDefault();  
  
    let name = document.getElementById("name").value;  
  
    if (name === "") {  
        alert("Name is required");  
        return;  
    }  
  
    console.log("Valid input");  
});
```

Email Validation

```
let email = document.getElementById("email").value;

if (!email.includes("@")) {
    alert("Invalid email");
}
```

Using Regular Expression (Advanced)

```
let emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;

if (!emailPattern.test(email)) {
    alert("Invalid email format");
}
```

Password Validation

```
let password = document.getElementById("password").value;

if (password.length < 6) {
    alert("Password must be at least 6 characters");
}
```

Display Error Messages

```
<p id="error"></p>
let error = document.getElementById("error");
```

```
if (name === "") {  
    error.innerText = "Name is required";  
}
```

Real-Time Validation (Input Event)

```
let input = document.getElementById("name");  
  
input.addEventListener("input", function() {  
    if (input.value.length < 3) {  
        console.log("Too short");  
    }  
});
```

Form Reset

```
form.reset();
```

Real-World Example

```
<form id="loginForm">  
    <input type="text" id="username">  
    <input type="password" id="password">  
    <button type="submit">Login</button>  
</form>  
  
<script>
```

```
document.getElementById("loginForm").addEventListener("submit", function(e) {
    e.preventDefault();

    let user = document.getElementById("username").value;
    let pass = document.getElementById("password").value;

    if (user === "" || pass === "") {
        alert("All fields required");
        return;
    }

    console.log("Login successful");
});
</script>
```

HTML5 Validation (Built-in)

<input type="email" required>

Features:

- required
- minlength
- maxlength
- pattern

Preventing Invalid Input

```
if (isNaN(age)) {  
    alert("Enter valid number");  
}
```

Best Practices

- Validate on both client and server
- Show clear error messages
- Avoid long forms
- Use proper input types
- Use real-time validation

Common Mistakes

1. Not using preventDefault
2. Not validating inputs
3. Poor error messages
4. Relying only on frontend validation
5. Ignoring edge cases

Interview Questions

1. What is form handling?
2. Why use preventDefault?
3. How to validate email?
4. What is regex?
5. Difference between client-side and server-side validation?

6. How to handle form submission?

Practice Exercises

1. Create form with validation
2. Validate email and password
3. Display error messages
4. Use regex
5. Implement real-time validation

Mini Practice Task

Create form:

- Name and email
- Validate both fields
- Show error if empty

Challenge Task

Create registration form:

- Username
- Email
- Password
- Confirm password

Validate all fields properly

CHAPTER 28: LOCAL STORAGE & SESSION STORAGE

Introduction

Local Storage and Session Storage are browser features used to store data on the client side. They allow web applications to save data in the browser so it can be used later without needing a server.

Why Use Browser Storage

- Store user preferences
- Save login state
- Store cart data
- Persist data across page reloads

Types of Storage

1. Local Storage
2. Session Storage

Local Storage

- Stores data permanently (until manually cleared)
- Shared across tabs of same origin

Storing Data

```
localStorage.setItem("name", "Ali");
```

Retrieving Data

```
let name = localStorage.getItem("name");  
console.log(name);
```

Output:

Ali

Removing Data

```
localStorage.removeItem("name");
```

Clearing All Data

```
localStorage.clear();
```

Session Storage

- Stores data temporarily
- Data is removed when tab is closed

Example

```
sessionStorage.setItem("user", "Ali");

let user = sessionStorage.getItem("user");
console.log(user);
```

Difference Between Local and Session Storage

Feature	Local Storage	Session Storage
Lifetime	Permanent	Until tab closes
Scope	All tabs	Single tab
Use Case	Long-term storage	Temporary data

Storing Objects (Important)

Storage only supports strings. Use JSON.

```
let user = { name: "Ali", age: 25 };

localStorage.setItem("user", JSON.stringify(user));
```

Retrieving Object

```
let data = localStorage.getItem("user");

let user = JSON.parse(data);

console.log(user.name);
```

Output:

Ali

Real-World Example: Todo App Storage

```
let tasks = ["Task1", "Task2"];

localStorage.setItem("tasks", JSON.stringify(tasks));

let storedTasks = JSON.parse(localStorage.getItem("tasks"));

console.log(storedTasks);
```

Example: Save Form Data

```
document.getElementById("name").addEventListener("input",
function(e) {
    localStorage.setItem("username", e.target.value);
});
```

Loading Stored Data

```
let savedName = localStorage.getItem("username");

if (savedName) {
    document.getElementById("name").value = savedName;
}
```

Storage Limit

- Usually around 5MB per domain
- Depends on browser

Best Practices

- Use JSON for complex data
- Clear unused data
- Avoid storing sensitive information
- Check for null before using data

Common Mistakes

1. Forgetting `JSON.stringify`
2. Not parsing JSON data
3. Storing sensitive data
4. Overwriting data unintentionally
5. Not checking if data exists

Interview Questions

1. What is `localStorage`?
2. Difference between `localStorage` and `sessionStorage`?
3. How to store objects in `localStorage`?
4. What is `JSON.stringify`?
5. What is `JSON.parse`?
6. Is `localStorage` secure?

Practice Exercises

1. Store and retrieve simple value
2. Store object using JSON
3. Remove specific item
4. Use sessionStorage
5. Build small storage system

Mini Practice Task

Create program:

- Store username
- Retrieve and display on reload

Challenge Task

Enhance Todo App:

- Store tasks in localStorage
- Load tasks on page refresh

CHAPTER 29: DEBUGGING TECHNIQUES

Introduction

Debugging is the process of finding and fixing errors in a program. Every developer spends a significant amount of time debugging code.

Good debugging skills help in:

- Identifying errors quickly
- Understanding program behavior
- Writing better code

Types of Bugs

1. Syntax Errors
2. Runtime Errors
3. Logical Errors

Using `console.log()`

The most basic debugging method.

```
let x = 10;  
console.log(x);
```

Debugging Example

```
let a = 5;
let b = 10;

console.log("a:", a);
console.log("b:", b);
console.log("sum:", a + b);
```

Console Methods

console.log()

```
console.log("Message");
```

console.error()

```
console.error("Error message");
```

console.warn()

```
console.warn("Warning message");
```

console.table()

```
let users = [
  { name: "Ali", age: 25 },
```

```
    { name: "Sara", age: 30 }  
  ];  
  
console.table(users);
```

Using Breakpoints

Steps:

1. Open DevTools (F12)
2. Go to Sources tab
3. Click on line number
4. Code pauses at that line

Step-by-Step Execution

- Step Over → next line
- Step Into → inside function
- Step Out → exit function

Using debugger Keyword

```
let x = 10;  
debugger;  
x += 5;  
console.log(x);
```

Explanation:

Execution pauses at debugger line.

Inspecting Variables

While paused:

- Hover over variables
- Check values
- Modify values

Network Debugging

- Go to Network tab
- Check API requests
- Inspect response

Debugging DOM Issues

Common checks:

```
console.log(document.getElementById("id"));
```

If null → element not found

Debugging Events

```
button.addEventListener("click", function() {  
    console.log("Clicked");  
});
```

```
});
```

Debugging Async Code

```
async function test() {  
  console.log("Start");  
  
  let data = await fetch("url");  
  console.log(data);  
  
  console.log("End");  
}
```

Real-World Example

```
function calculateTotal(price, quantity) {  
  console.log("Price:", price);  
  console.log("Quantity:", quantity);  
  
  let total = price * quantity;  
  
  console.log("Total:", total);  
  
  return total;  
}  
  
calculateTotal(100, 2);
```

Debugging Strategies

- Check error message carefully
- Use console logs at key points
- Break problem into small parts
- Test with simple inputs
- Verify assumptions

Common Mistakes

1. Ignoring console errors
2. Not checking null values
3. Misplacing console.log
4. Debugging without understanding flow
5. Not using DevTools properly

Interview Questions

1. What is debugging?
2. How do you debug JavaScript code?
3. What is breakpoint?
4. What is debugger keyword?
5. How to debug API calls?
6. What are console methods?

Practice Exercises

1. Use console.log to debug
2. Add breakpoint and inspect values
3. Use debugger keyword

4. Debug DOM selection issue
5. Debug API response

Mini Practice Task

Create program:

- Write function with bug
- Use console to find error

Challenge Task

Create app:

- Fetch API data
- Add intentional error
- Debug and fix it

CHAPTER 30: CALCULATOR PROJECT

Introduction

This project builds a basic calculator using HTML, CSS, and JavaScript. It demonstrates:

- DOM manipulation
- Event handling
- Working with user input
- Implementing logic

This is a common beginner project and useful for interviews.

Problem Statement

Create a calculator that can:

- Perform addition, subtraction, multiplication, division
- Take input from buttons
- Display result

Project Structure

```
project/  
|  
├── index.html  
├── style.css  
└── script.js
```


Step 1: HTML Structure

```
<!DOCTYPE html>
<html>
<head>
  <title>Calculator</title>
</head>
<body>

<h2>Calculator</h2>

<input type="text" id="display" readonly>

<br>

<button onclick="appendValue('7')">7</button>
<button onclick="appendValue('8')">8</button>
<button onclick="appendValue('9')">9</button>
<button onclick="appendValue('/')">/</button>

<br>

<button onclick="appendValue('4')">4</button>
<button onclick="appendValue('5')">5</button>
<button onclick="appendValue('6')">6</button>
<button onclick="appendValue('*')">*</button>

<br>

<button onclick="appendValue('1')">1</button>
<button onclick="appendValue('2')">2</button>
<button onclick="appendValue('3')">3</button>
<button onclick="appendValue('-')">-</button>
```

```
<br>

<button onclick="appendValue('0')">0</button>
<button onclick="clearDisplay()">C</button>
<button onclick="calculate()">=</button>
<button onclick="appendValue('+')">+</button>

<script src="script.js"></script>

</body>
</html>
```

Step 2: JavaScript Logic

```
let display = document.getElementById("display");

function appendValue(value) {
    display.value += value;
}

function clearDisplay() {
    display.value = "";
}

function calculate() {
    try {
        display.value = eval(display.value);
    } catch {
        display.value = "Error";
    }
}
```

Explanation

- appendValue → adds numbers/operators
- clearDisplay → clears input
- calculate → evaluates expression

Output

User clicks buttons → values appear in display

Press "=" → result is shown

Step 3: Basic Styling (Optional)

```
body {  
    font-family: Arial;  
}
```

```
button {  
    width: 50px;  
    height: 40px;  
    margin: 5px;  
}
```

Improved Version (Without eval)

Using safer approach:

```
function calculate() {  
    let exp = display.value;  
  
    let result = Function("return " + exp)();  
    display.value = result;  
}
```

Adding Keyboard Support

```
document.addEventListener("keydown", function(e) {  
    if (!isNaN(e.key) || "+-*/*.".includes(e.key)) {  
        appendValue(e.key);  
    }  
  
    if (e.key === "Enter") {  
        calculate();  
    }  
  
    if (e.key === "Backspace") {  
        display.value = display.value.slice(0, -1);  
    }  
});
```

Real-World Enhancements

- Add decimal support
- Add history
- Add scientific functions
- Add UI improvements

Common Mistakes

1. Using eval without validation
2. Not handling invalid expressions
3. Not clearing display properly
4. Ignoring keyboard input
5. Poor UI layout

Interview Questions

1. How does calculator logic work?
2. Why is eval risky?
3. How to handle invalid input?
4. How to improve calculator?
5. How to add keyboard support?

Practice Exercises

1. Add decimal support
2. Add backspace button
3. Add square root function
4. Improve UI
5. Prevent invalid input

Mini Practice Task

Modify calculator:

- Add "%" operator

- Handle percentage

Challenge Task

Build advanced calculator:

- History tracking
- Dark mode
- Keyboard support
- Error handling

CHAPTER 31: TODO APP (ADVANCED WITH LOCAL STORAGE)

Introduction

This chapter upgrades the basic Todo App by adding persistent storage using `localStorage`. Now tasks will remain saved even after page reload.

This project demonstrates:

- DOM manipulation
- Event handling
- Local storage usage
- Data persistence

Problem Statement

Create a Todo App where user can:

- Add tasks
- Delete tasks
- Mark tasks as completed
- Save tasks permanently

Project Structure

```
project/  
|  
└─ index.html
```

- └─ style.css
- └─ script.js

Step 1: HTML Structure

```
<!DOCTYPE html>
<html>
<head>
  <title>Todo App</title>
</head>
<body>

<h2>Todo App</h2>

<input type="text" id="taskInput" placeholder="Enter task">
<button id="addBtn">Add Task</button>

<ul id="taskList"></ul>

<script src="script.js"></script>

</body>
</html>
```

Step 2: JavaScript Setup

```
let input = document.getElementById("taskInput");
let button = document.getElementById("addBtn");
let list = document.getElementById("taskList");
```



```
let tasks = JSON.parse(localStorage.getItem("tasks")) || [];
```

Step 3: Save Tasks to Local Storage

```
function saveTasks() {  
    localStorage.setItem("tasks", JSON.stringify(tasks));  
}
```

Step 4: Render Tasks

```
function renderTasks() {  
    list.innerHTML = "";  
  
    tasks.forEach((task, index) => {  
        let li = document.createElement("li");  
        li.innerText = task.text;  
  
        if (task.completed) {  
            li.style.textDecoration = "line-through";  
        }  
  
        li.addEventListener("click", function() {  
            tasks[index].completed  
= !tasks[index].completed;  
            saveTasks();  
            renderTasks();  
        }));  
  
        let deleteBtn = document.createElement("button");  
        deleteBtn.innerText = "Delete";
```

```

        deleteBtn.addEventListener("click", function() {
            tasks.splice(index, 1);
            saveTasks();
            renderTasks();
        });

        li.appendChild(deleteBtn);
        list.appendChild(li);
    });
}

```

Step 5: Add Task

```

button.addEventListener("click", function() {
    let taskText = input.value;

    if (taskText === "") return;

    tasks.push({
        text: taskText,
        completed: false
    });

    saveTasks();
    renderTasks();

    input.value = "";
});

```

Step 6: Load Tasks on Page Load

```
renderTasks();
```

Explanation

- tasks array stores all tasks
- localStorage stores data permanently
- renderTasks updates UI
- saveTasks saves data

Output

- Tasks remain after page reload
- Tasks can be marked complete
- Tasks can be deleted

Full Combined Code

```
let input = document.getElementById("taskInput");
let button = document.getElementById("addBtn");
let list = document.getElementById("taskList");

let tasks = JSON.parse(localStorage.getItem("tasks")) || [];

function saveTasks() {
    localStorage.setItem("tasks", JSON.stringify(tasks));
}
```

```

function renderTasks() {
    list.innerHTML = "";

    tasks.forEach((task, index) => {
        let li = document.createElement("li");
        li.innerText = task.text;

        if (task.completed) {
            li.style.textDecoration = "line-through";
        }

        li.addEventListener("click", function() {
            tasks[index].completed = !task.completed;
            saveTasks();
            renderTasks();
        });

        let deleteBtn = document.createElement("button");
        deleteBtn.innerText = "Delete";

        deleteBtn.addEventListener("click", function() {
            tasks.splice(index, 1);
            saveTasks();
            renderTasks();
        });

        li.appendChild(deleteBtn);
        list.appendChild(li);
    });
}

button.addEventListener("click", function() {
    let taskText = input.value;

    if (taskText === "") return;

```

```
tasks.push({
  text: taskText,
  completed: false
});

saveTasks();
renderTasks();

input.value = "";
});

renderTasks();
```

Real-World Enhancements

- Add edit task feature
- Add filters (completed / pending)
- Add due dates
- Add drag and drop
- Add categories

Common Mistakes

1. Forgetting JSON.parse / stringify
2. Not updating UI after change
3. Incorrect indexing
4. Overwriting localStorage
5. Not handling empty input

Interview Questions

1. How does localStorage work?
2. How to persist data in browser?
3. How to update UI dynamically?
4. What is JSON.stringify?
5. How to handle state in JavaScript?

Practice Exercises

1. Add edit feature
2. Add clear all button
3. Add task count
4. Add completed filter
5. Improve UI

Mini Practice Task

Add feature:

- Show total tasks count

Challenge Task

Build full Todo App:

- Local storage
- Filters
- Edit task
- Responsive UI

CHAPTER 32: WEATHER APP (API BASED)

Introduction

This project builds a Weather App using JavaScript and an external API. It demonstrates:

- Fetch API usage
- Async/Await
- DOM manipulation
- Handling user input
- Displaying real-time data

This is a very common real-world and interview project.

Problem Statement

Create a Weather App where a user can:

- Enter a city name
- Fetch weather data from API
- Display temperature and condition

Project Structure

```
project/  
|  
└─ index.html
```


- └─ style.css
- └─ script.js

Step 1: HTML Structure

```
<!DOCTYPE html>
<html>
<head>
  <title>Weather App</title>
</head>
<body>

<h2>Weather App</h2>

<input type="text" id="city" placeholder="Enter city">
<button id="btn">Get Weather</button>

<p id="result"></p>

<script src="script.js"></script>

</body>
</html>
```

Step 2: API Setup

Use a public weather API (example structure):

<https://api.example.com/weather?city=Delhi>

Response example:

```
{
  "temp": 25,
  "condition": "Cloudy"
}
```

Step 3: JavaScript Logic

```
let button = document.getElementById("btn");

button.addEventListener("click", async function() {
  let city = document.getElementById("city").value;

  if (city === "") {
    alert("Enter city name");
    return;
  }

  try {
    let response = await
fetch(`https://api.example.com/weather?city=${city}`);

    if (!response.ok) {
      throw new Error("City not found");
    }

    let data = await response.json();

    document.getElementById("result").innerText =
      `Temperature: ${data.temp}°C, Condition:
${data.condition}`;
  }
}
```

```
    } catch (error) {  
        document.getElementById("result").innerText =  
error.message;  
    }  
});
```

Explanation

- Get city input
- Send API request
- Convert response to JSON
- Display result
- Handle errors

Output

User enters "Delhi":

Temperature: 25°C, Condition: Cloudy

Step 4: Improve UI (Optional)

```
body {  
    font-family: Arial;  
    text-align: center;  
}  
  
input {  
    padding: 5px;
```

```
}  
  
button {  
    padding: 5px 10px;  
}
```

Using Real API (Example: OpenWeather)

```
let apiKey = "YOUR_API_KEY";  
  
let url =  
`https://api.openweathermap.org/data/2.5/weather?q=${city}&a  
ppid=${apiKey}&units=metric`;
```

Extract Data

```
let temp = data.main.temp;  
let condition = data.weather[0].main;
```

Real-World Example

```
document.getElementById("btn").addEventListener("click",  
async function() {  
    let city = document.getElementById("city").value;  
  
    let apiKey = "YOUR_API_KEY";  
  
    let url =
```

```
`https://api.openweathermap.org/data/2.5/weather?q=${city}&appid=${apiKey}&units=metric`;
```

```
    try {  
        let response = await fetch(url);  
        let data = await response.json();  
  
        document.getElementById("result").innerText =  
            `${data.name}: ${data.main.temp}°C,  
            ${data.weather[0].main}`;  
  
    } catch {  
        document.getElementById("result").innerText = "Error  
fetching data";  
    }  
});
```

Real-World Enhancements

- Add weather icons
- Show humidity and wind speed
- Add loading spinner
- Auto-detect location
- Add forecast (5-day)

Common Mistakes

1. Forgetting API key
2. Wrong API URL
3. Not handling errors
4. Not checking response.ok

5. Not parsing JSON correctly

Interview Questions

1. How does Fetch API work in this project?
2. What is async/await?
3. How do you handle API errors?
4. What is JSON response?
5. How to improve this app?

Practice Exercises

1. Add humidity display
2. Add wind speed
3. Add loading message
4. Improve UI
5. Handle invalid city

Mini Practice Task

Modify app:

- Show only temperature

Challenge Task

Build advanced weather app:

- Show 5-day forecast

- Add icons
- Save last searched city

CHAPTER 33: MINI E-COMMERCE UI

Introduction

This project builds a basic e-commerce interface using JavaScript. It demonstrates:

- Rendering products dynamically
- Managing cart functionality
- Updating UI in real-time
- Handling user interactions

This is a practical project often expected in frontend roles.

Problem Statement

Create a Mini E-commerce UI where a user can:

- View products
- Add products to cart
- Remove products from cart
- See total price

Project Structure

```
project/  
|  
├── index.html  
├── style.css  
└── script.js
```


Step 1: HTML Structure

```
<!DOCTYPE html>
<html>
<head>
  <title>E-commerce</title>
</head>
<body>

  <h2>Products</h2>
  <div id="productList"></div>

  <h2>Cart</h2>
  <ul id="cartList"></ul>
  <p id="total"></p>

  <script src="script.js"></script>

</body>
</html>
```

Step 2: Product Data

```
let products = [
  { id: 1, name: "Laptop", price: 50000 },
  { id: 2, name: "Phone", price: 20000 },
  { id: 3, name: "Headphones", price: 2000 }
];

let cart = [];
```

Step 3: Display Products

```
let productList = document.getElementById("productList");

function renderProducts() {
  products.forEach(product => {
    let div = document.createElement("div");

    div.innerHTML = `
      ${product.name} - ₹${product.price}
      <button
onclick="addToCart(${product.id})">Add</button>
    `;

    productList.appendChild(div);
  });
}

renderProducts();
```

Step 4: Add to Cart

```
function addToCart(id) {
  let product = products.find(p => p.id === id);
  cart.push(product);
  renderCart();
}
```

Step 5: Display Cart

```
let cartList = document.getElementById("cartList");
let totalEl = document.getElementById("total");

function renderCart() {
    cartList.innerHTML = "";

    let total = 0;

    cart.forEach((item, index) => {
        let li = document.createElement("li");

        li.innerHTML = `
            ${item.name} - ₹${item.price}
            <button
onclick="removeFromCart(${index})">Remove</button>
        `;

        cartList.appendChild(li);

        total += item.price;
    });

    totalEl.innerText = "Total: ₹" + total;
}
```

Step 6: Remove from Cart

```
function removeFromCart(index) {
    cart.splice(index, 1);
    renderCart();
}
```

```
}
```

Explanation

- products array stores items
- cart array stores selected items
- renderProducts shows products
- renderCart updates cart UI and total

Output

- Products displayed with Add button
- Cart updates dynamically
- Total price updates

Full Combined Code

```
let products = [  
  { id: 1, name: "Laptop", price: 50000 },  
  { id: 2, name: "Phone", price: 20000 },  
  { id: 3, name: "Headphones", price: 2000 }  
];  
  
let cart = [];  
  
let productList = document.getElementById("productList");  
let cartList = document.getElementById("cartList");  
let totalEl = document.getElementById("total");
```

```

function renderProducts() {
    productList.innerHTML = "";

    products.forEach(product => {
        let div = document.createElement("div");

        div.innerHTML = `
            ${product.name} - ₹${product.price}
            <button
onclick="addToCart(${product.id})">Add</button>
        `;

        productList.appendChild(div);
    });
}

function addToCart(id) {
    let product = products.find(p => p.id === id);
    cart.push(product);
    renderCart();
}

function removeFromCart(index) {
    cart.splice(index, 1);
    renderCart();
}

function renderCart() {
    cartList.innerHTML = "";

    let total = 0;

    cart.forEach((item, index) => {
        let li = document.createElement("li");

```

```

        li.innerHTML = `
            ${item.name} - ₹${item.price}
            <button
onclick="removeFromCart(${index})">Remove</button>
        `;

        cartList.appendChild(li);

        total += item.price;
    });

    totalEl.innerText = "Total: ₹" + total;
}

renderProducts();

```

Real-World Enhancements

- Add quantity feature
- Store cart in localStorage
- Add product images
- Add search/filter
- Add checkout system

Common Mistakes

1. Not updating UI after cart change
2. Incorrect indexing in remove
3. Forgetting to clear previous elements
4. Not calculating total properly
5. Direct DOM manipulation without structure

Interview Questions

1. How does cart system work?
2. How to manage state in JavaScript?
3. How to optimize rendering?
4. How to persist cart data?
5. How to improve this project?

Practice Exercises

1. Add quantity to cart
2. Store cart in localStorage
3. Add product images
4. Add search feature
5. Improve UI

Mini Practice Task

Add feature:

- Show number of items in cart

Challenge Task

Build advanced e-commerce UI:

- Add product filtering
- Add cart persistence
- Add checkout page

CHAPTER 34: JAVASCRIPT INTERVIEW QUESTIONS

Introduction

This chapter covers commonly asked JavaScript interview questions. These include conceptual, output-based, and scenario-based questions that are frequently asked in frontend and full-stack interviews.

Basic Concept Questions

1. What is JavaScript

JavaScript is a high-level, interpreted programming language used to create dynamic and interactive web applications. It runs in the browser and can also run on the server using Node.js.

2. Difference between var, let, const

Feature	var	let	const
Scope	Function	Block	Block
Redeclare	Yes	No	No
Reassign	Yes	Yes	No

3. What is closure

A closure is a function that remembers variables from its outer scope even after the outer function has finished execution.

4. What is hoisting

Hoisting is JavaScript behavior where variable and function declarations are moved to the top of their scope before execution.

5. What is event loop

Event loop handles asynchronous operations and ensures non-blocking execution in JavaScript.

6. Difference between == and ===

- == compares value only
- === compares value and type

7. What is DOM

DOM is a representation of HTML document that allows JavaScript to access and modify elements.

Output-Based Questions

1.

```
console.log(5 + "5");
```

Output:

"55"

2.

```
console.log(typeof null);
```

Output:

object

3.

```
console.log(0 == false);  
console.log(0 === false);
```

Output:

true

false

4.

```
let x;  
console.log(x);
```

Output:

undefined

5.

```
console.log([] + []);
```

Output:

"" (empty string)

6.

```
console.log([] + {});
```

Output:

"[object Object]"

Function-Based Questions

1. What is a callback function

A function passed as an argument to another function.

2. What are arrow functions

Short syntax functions introduced in ES6.

3. Difference between function declaration and expression

- Declaration is hoisted
- Expression is not hoisted

4. What is IIFE

Immediately Invoked Function Expression executes immediately after definition.

Asynchronous JavaScript Questions

1. What is Promise

An object that represents future completion of an async operation.

2. What is async/await

A cleaner way to handle promises using synchronous-like syntax.

3. What is callback hell

Nested callbacks making code unreadable.

4. Difference between synchronous and asynchronous

- Synchronous → line by line
- Asynchronous → non-blocking

DOM & Event Questions

1. What is event bubbling

Event moves from child to parent.

2. What is event delegation

Handling events on parent element instead of child.

3. Difference between `querySelector` and `getElementById`

- `querySelector` uses CSS selectors
- `getElementById` uses id only

Object & Array Questions

1. Difference between `map` and `forEach`

- `map` returns new array
- `forEach` does not return

2. What is destructuring

Extract values from arrays or objects into variables.

3. What is spread operator

Used to expand elements.

4. What is rest operator

Used to collect multiple values.

Advanced Questions

1. What is prototype

Mechanism for inheritance in JavaScript.

2. What is this keyword

Refers to current object.

3. What is class in JavaScript

Blueprint for creating objects.

4. What is module

A file containing reusable code.

Scenario-Based Questions

1. How to handle API errors

Use try-catch with async/await and check response.ok.

2. How to optimize performance

- Reduce DOM manipulation
- Use caching

- Optimize loops

3. How to manage state in JavaScript

- Use variables
- Use localStorage
- Use frameworks

4. How to debug JavaScript

- Use console.log
- Use DevTools
- Use breakpoints

Coding Questions

Reverse String

```
function reverse(str) {  
    return str.split("").reverse().join("");  
}
```

Check Palindrome

```
function isPalindrome(str) {  
    let reversed = str.split("").reverse().join("");  
    return str === reversed;  
}
```


Find Largest Number

```
function max(arr) {  
    return Math.max(...arr);  
}
```

Common Mistakes in Interviews

- Not understanding basics
- Writing incorrect syntax
- Not explaining logic
- Ignoring edge cases
- Poor debugging skills

Practice Tips

- Practice output questions daily
- Write code manually
- Explain concepts clearly
- Build projects
- Revise fundamentals

Mini Practice Task

Explain:

- Closure
- Promise

- Event loop

Challenge Task

Solve:

- Reverse string
- Find duplicates in array
- Implement simple API call

CHAPTER 35: CODING PROBLEMS

Introduction

This chapter contains important coding problems frequently asked in interviews. These problems help build logic, problem-solving ability, and coding confidence.

Each problem includes:

- Problem statement
- Approach
- Code
- Explanation

Problem 1: Reverse a String

Problem

Reverse a given string.

Approach

- Convert string to array
- Reverse array
- Join back to string

Code

```
function reverseString(str) {  
    return str.split("").reverse().join("");  
}  
  
console.log(reverseString("hello"));
```

Output

olleh

Explanation

- `split("")` → converts string to array
- `reverse()` → reverses array
- `join("")` → converts back to string

Problem 2: Check Palindrome

Problem

Check if a string is palindrome.

Code

```
function isPalindrome(str) {  
    let reversed = str.split("").reverse().join("");  
    return str === reversed;  
}
```

```
}  
  
console.log(isPalindrome("madam"));
```

Output

true

Problem 3: Find Maximum Number in Array

Code

```
function findMax(arr) {  
    return Math.max(...arr);  
}  
  
console.log(findMax([1, 5, 3, 9]));
```

Output

9

Problem 4: Find Minimum Number

```
function findMin(arr) {  
    return Math.min(...arr);  
}
```

Problem 5: Sum of Array

```
function sumArray(arr) {  
    return arr.reduce((sum, num) => sum + num, 0);  
}
```

Problem 6: Count Vowels

```
function countVowels(str) {  
    let count = 0;  
    let vowels = "aeiou";  
  
    for (let char of str.toLowerCase()) {  
        if (vowels.includes(char)) {  
            count++;  
        }  
    }  
  
    return count;  
}
```

Problem 7: Remove Duplicates from Array

```
function removeDuplicates(arr) {  
    return [...new Set(arr)];  
}
```

Problem 8: Factorial

```
function factorial(n) {  
    if (n === 0) return 1;  
    return n * factorial(n - 1);  
}
```

Problem 9: Check Even or Odd

```
function isEven(n) {  
    return n % 2 === 0;  
}
```

Problem 10: FizzBuzz

```
function fizzBuzz(n) {  
    for (let i = 1; i <= n; i++) {  
        if (i % 3 === 0 && i % 5 === 0)  
            console.log("FizzBuzz");  
        else if (i % 3 === 0) console.log("Fizz");  
        else if (i % 5 === 0) console.log("Buzz");  
        else console.log(i);  
    }  
}
```

Problem 11: Find Duplicate Elements

```
function findDuplicates(arr) {  
  let seen = new Set();  
  let duplicates = [];  
  
  for (let num of arr) {  
    if (seen.has(num)) {  
      duplicates.push(num);  
    } else {  
      seen.add(num);  
    }  
  }  
  
  return duplicates;  
}
```

Problem 12: Sort Array

```
function sortArray(arr) {  
  return arr.sort((a, b) => a - b);  
}
```

Problem 13: Capitalize First Letter

```
function capitalize(str) {  
  return str.charAt(0).toUpperCase() + str.slice(1);  
}
```


Problem 14: Count Occurrences

```
function countOccurrences(arr, value) {  
    return arr.filter(x => x === value).length;  
}
```

Problem 15: Flatten Array

```
function flatten(arr) {  
    return arr.flat();  
}
```

Real-World Example

```
let prices = [100, 200, 300];  
  
let total = prices.reduce((sum, price) => sum + price, 0);  
  
console.log(total);
```

Common Mistakes

1. Not handling edge cases
2. Writing inefficient code
3. Forgetting return statement
4. Using wrong data structures

5. Not understanding problem properly

Interview Questions

1. How to optimize array operations?
2. Difference between map, filter, reduce?
3. How to remove duplicates?
4. How to reverse string?
5. How to handle edge cases?

Practice Exercises

1. Reverse words in sentence
2. Find second largest number
3. Check anagram
4. Merge two arrays
5. Find missing number

Mini Practice Task

Solve:

- Reverse string
- Find max
- Remove duplicates

Challenge Task

Solve:

- Implement custom map function
- Implement custom reduce
- Build mini problem solver

CHAPTER 36: SCENARIO-BASED QUESTIONS

Introduction

Scenario-based questions test your practical understanding of JavaScript. Instead of theory, you are given real-world situations and asked how you would solve them.

These questions are commonly asked in interviews to evaluate:

- Problem-solving ability
- Real-world thinking
- Code design skills

Scenario 1: Form Not Submitting Correctly

Problem

Form reloads page and data is not processed.

Solution

```
form.addEventListener("submit", function(event) {  
    event.preventDefault();  
    console.log("Form handled");  
});
```

Explanation

preventDefault() stops page reload and allows custom handling.

Scenario 2: API Not Returning Data

Problem

Data is not showing after API call.

Solution

```
async function fetchData() {  
  try {  
    let response = await fetch("url");  
  
    if (!response.ok) {  
      throw new Error("API error");  
    }  
  
    let data = await response.json();  
    console.log(data);  
  
  } catch (error) {  
    console.log(error.message);  
  }  
}
```

Explanation

Check response.ok and handle errors properly.

Scenario 3: Button Click Not Working

Problem

Click event is not triggered.

Solution

```
document.getElementById("btn").addEventListener("click",  
function() {  
    console.log("Clicked");  
});
```

Possible Causes

- Wrong ID
- Script loaded before DOM
- Event not attached

Scenario 4: Slow Performance in Large List

Problem

Rendering many elements slows down page.

Solution

- Use document fragment

- Minimize DOM updates

```
let fragment = document.createDocumentFragment();

for (let i = 0; i < 1000; i++) {
  let li = document.createElement("li");
  li.innerText = i;
  fragment.appendChild(li);
}

list.appendChild(fragment);
```

Scenario 5: Duplicate Items in Array

Problem

Array contains duplicate values.

Solution

```
let unique = [...new Set(arr)];
```

Scenario 6: Data Lost After Page Reload

Problem

User data disappears after refresh.

Solution

Use localStorage:

```
localStorage.setItem("data", JSON.stringify(data));
```

Scenario 7: Infinite Loop Issue

Problem

Loop runs endlessly.

Solution

Check condition:

```
while (i < 10) {  
    i++;  
}
```

Scenario 8: Cannot Access Variable Outside Function

Problem

Variable not accessible.

Solution

Use proper scope:


```
function test() {  
    let x = 10;  
    return x;  
}  
  
let result = test();  
console.log(result);
```

Scenario 9: Handling Multiple Click Events

Problem

Many elements need same event.

Solution (Event Delegation)

```
document.getElementById("list").addEventListener("click",  
function(e) {  
    console.log(e.target.innerText);  
}));
```

Scenario 10: Form Validation Fails

Problem

Invalid data is submitted.

Solution

```
if (input.value === "") {  
    alert("Required");  
}
```

Scenario 11: Incorrect Output in Async Code

Problem

```
console.log("Start");  
  
setTimeout(() => console.log("Async"), 0);  
  
console.log("End");
```

Output

Start

End

Async

Explanation

Async code executes after synchronous code due to event loop.

Scenario 12: Need to Clone Object

Problem

Object changes affect original.

Solution

```
let copy = { ...original };
```

Scenario 13: Sorting Not Working Properly

Problem

```
[10, 2, 5].sort();
```

Output

```
[10, 2, 5]
```

Solution

```
[10, 2, 5].sort((a, b) => a - b);
```

Scenario 14: Cannot Handle API Errors

Solution

Use try-catch:

```
try {  
    let data = await fetch(url);  
} catch (e) {  
    console.log(e);  
}
```

Scenario 15: Need to Delay Execution

Solution

```
setTimeout(() => {  
    console.log("Delayed");  
}, 2000);
```

Real-World Scenario

Shopping Cart Issue

Problem: Total price incorrect

Solution:

```
let total = cart.reduce((sum, item) => sum + item.price, 0);
```

Common Mistakes

1. Ignoring error handling
2. Not understanding async flow
3. Poor DOM manipulation
4. Incorrect scope usage
5. Inefficient loops

Interview Questions

1. How to debug real-world issues?
2. How to optimize performance?
3. How to handle async problems?
4. How to manage large data?
5. How to design scalable code?

Practice Exercises

1. Fix a broken form
2. Debug API issue
3. Optimize loop
4. Handle duplicates
5. Improve UI performance

Mini Practice Task

Solve:

- Prevent form reload
- Fix event issue

Challenge Task

Build system:

- API integration
- Error handling
- Data persistence

END OF BOOK